

Chad Galloway
CST-250 Programming in C# II
Grand Canyon University
Nov. 09, 2025
Activity 3

Files

<https://github.com/CGalloway3/CST-250-Projects/tree/master/Activity3>

Video

<https://www.loom.com/share/6e68231da7924e208b7aa5dbfced8d52>

PART 1

Flow Chart

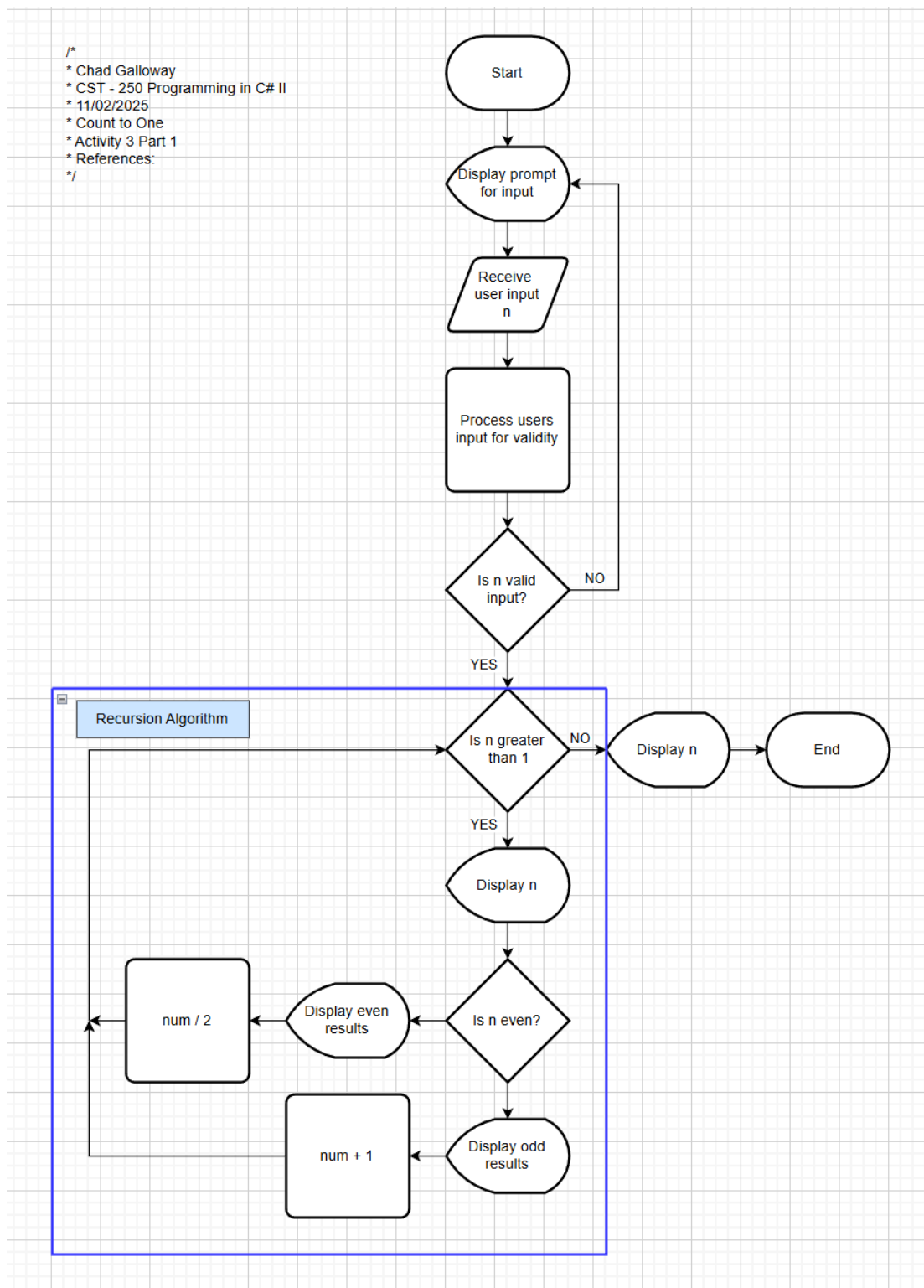


Figure 1-1: Flow chart of Activity 3 part one, count to one recursion console application.

UML Class Diagram

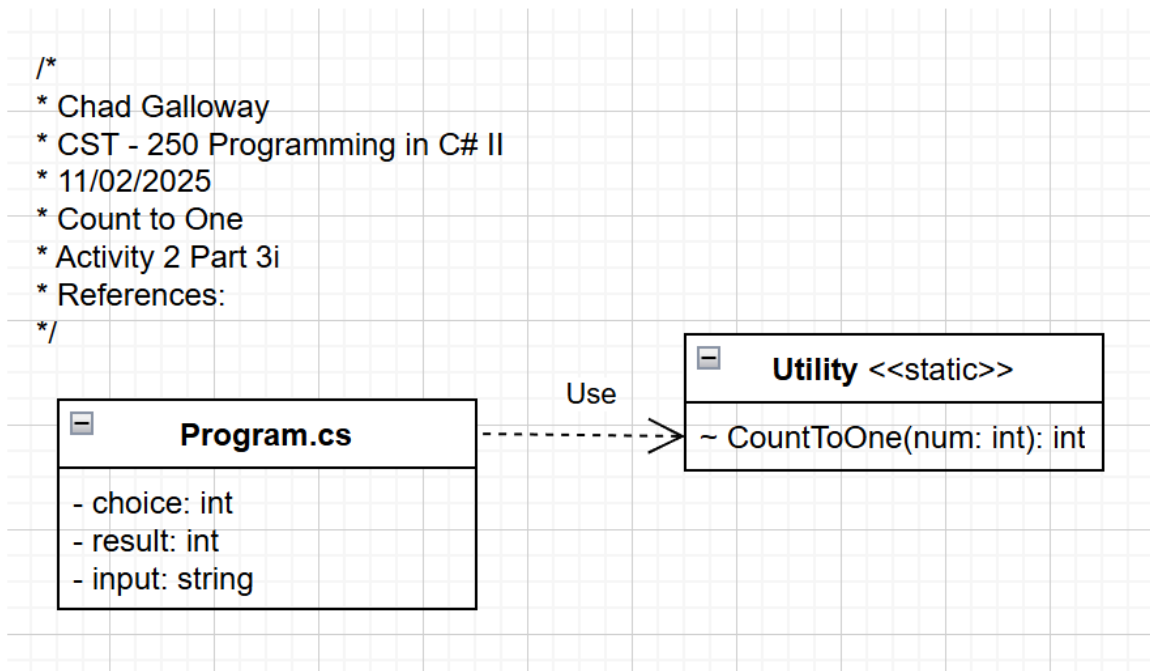


Figure 1-2: UML Class Diagram for Activity 3 part one, count to one recursion console application.

Screen Shots

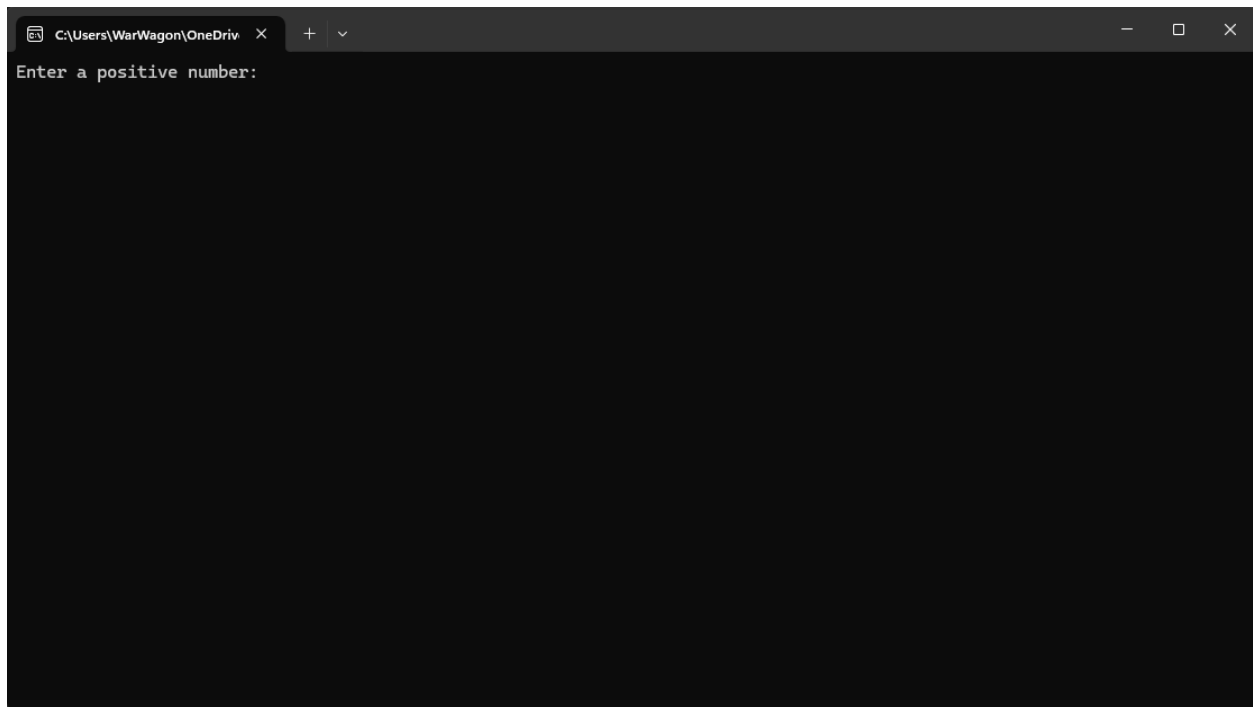
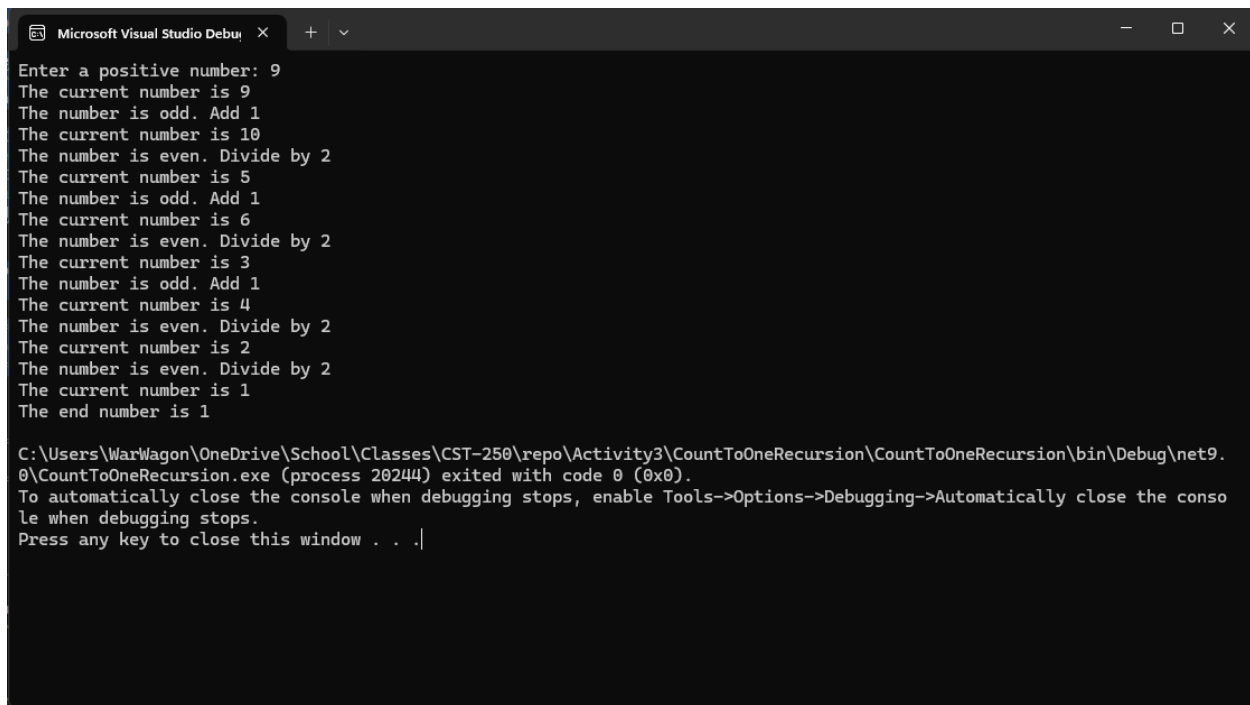


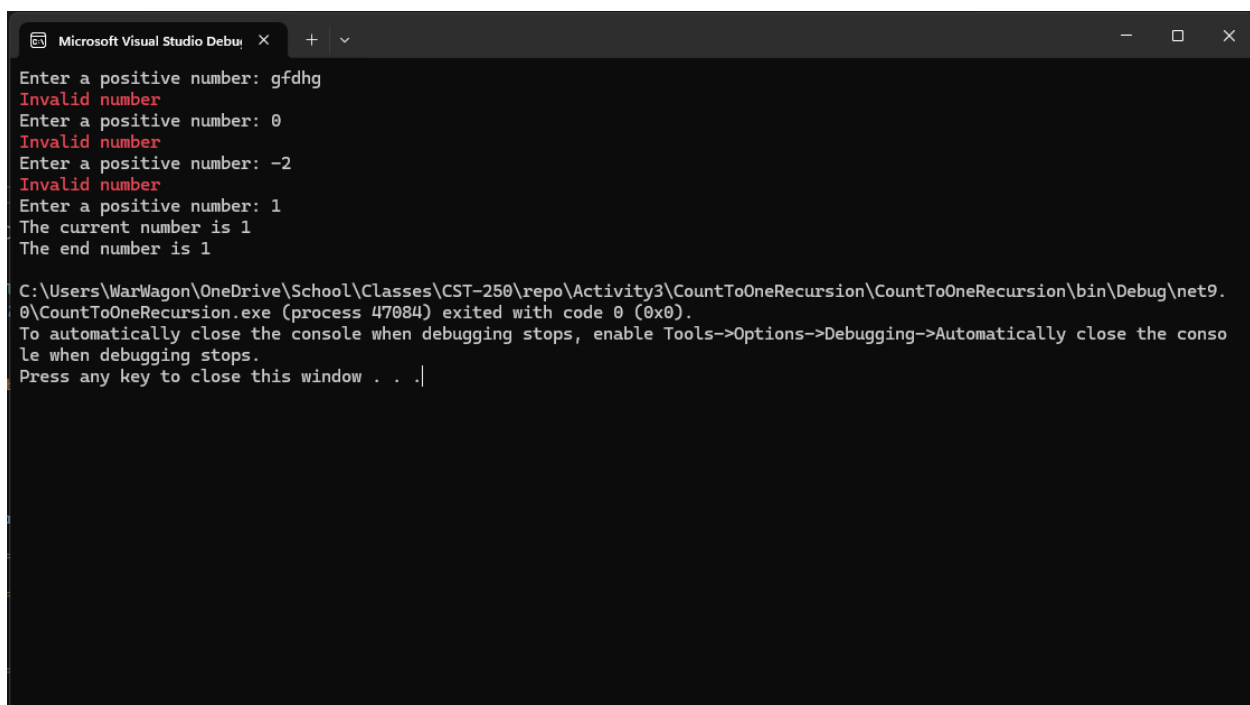
Figure 1-3: Part 1 App Running in Initial load state.



```
Microsoft Visual Studio Debug Console
Enter a positive number: 9
The current number is 9
The number is odd. Add 1
The current number is 10
The number is even. Divide by 2
The current number is 5
The number is odd. Add 1
The current number is 6
The number is even. Divide by 2
The current number is 3
The number is odd. Add 1
The current number is 4
The number is even. Divide by 2
The current number is 2
The number is even. Divide by 2
The current number is 1
The end number is 1

C:\Users\WarWagon\OneDrive\School\Classes\CST-250\repo\Activity3\CountToOneRecursion\CountToOneRecursion\bin\Debug\net9.0\CountToOneRecursion.exe (process 20244) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops.
Press any key to close this window . . .|
```

Figure 1-4: Screenshot of successful output



```
Microsoft Visual Studio Debug Console
Enter a positive number: gfdhg
Invalid number
Enter a positive number: 0
Invalid number
Enter a positive number: -2
Invalid number
Enter a positive number: 1
The current number is 1
The end number is 1

C:\Users\WarWagon\OneDrive\School\Classes\CST-250\repo\Activity3\CountToOneRecursion\CountToOneRecursion\bin\Debug\net9.0\CountToOneRecursion.exe (process 47084) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops.
Press any key to close this window . . .|
```

Figure 1-5: Screenshot of Error Handling

Code

```
Program.cs - X
[1] CountToOneRecursion - %Utility - %CountToOne(int num)
1  //
2  // * Chad Gallosay
3  // * C#7 - 2nd Programming in C# II
4  // * 11/29/2020
5  // * Count to one recursion
6  // * Activity 3
7  // * References:
8  //
9
10 //=====
11 // Start of the Main Method
12 //=====
13
14 // Declare and Initialize
15
16 using System.Web;
17
18 int choice = 0, result = 0;
19 string input = "";
20
21 // Prompt the user for a number
22 Console.WriteLine("Enter a positive number: ");
23 // Get the users input
24 input = Console.ReadLine();
25
26 // See if the user entered valid input
27 while (!int.TryParse(input, out choice) && choice > 0)
28 {
29     // Display error
30     Console.ForegroundColor = ConsoleColor.Red;
31     Console.WriteLine("Invalid number");
32     Console.ResetColor();
33
34     // Re-prompt the user for a number
35     Console.WriteLine("Enter a positive number: ");
36
37     // Get the user input
38     input = Console.ReadLine();
39 }
40
41 // Call the CountToOne
42 result = Utility.CountToOne(choice);
43 Console.WriteLine($"The end number is {result}");
44
45 //=====
46 // End of the Main Method
47 //=====
```

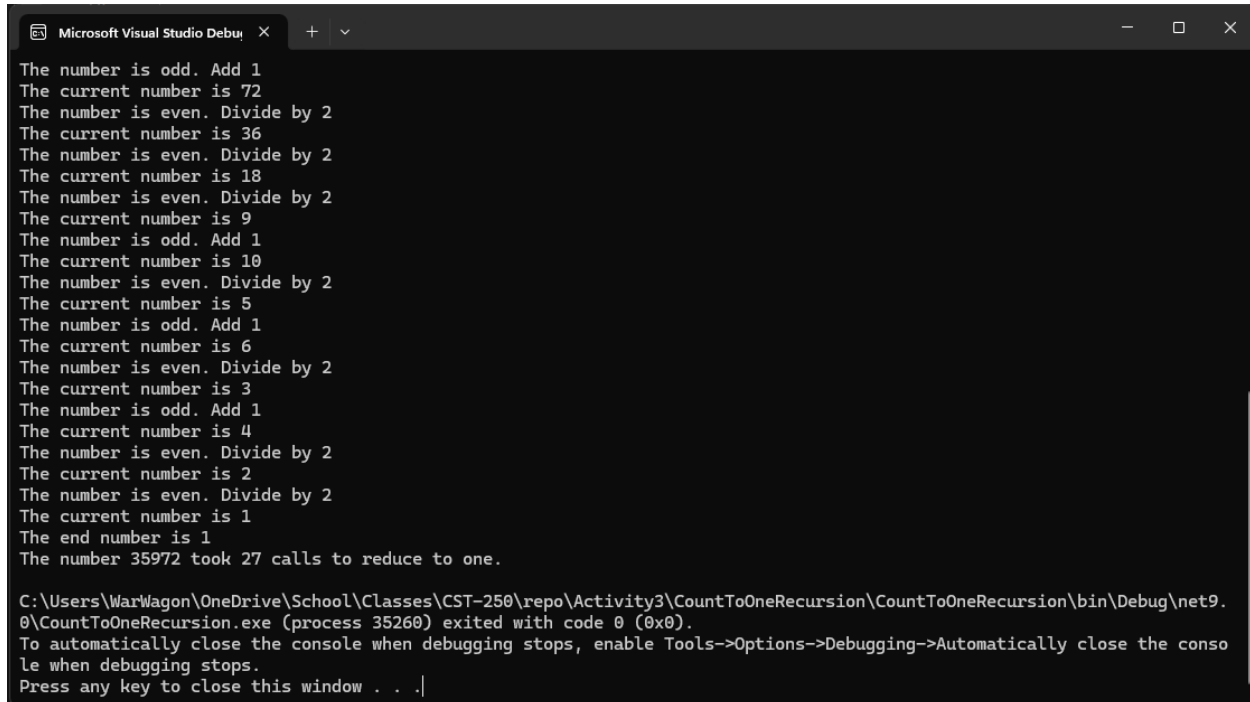
Figure 1-6: Code Citations and the main method.

```
Program.cs - X
[1] CountToOneRecursion - %Utility - %CountToOne(int num)
1  //
2  // * Chad Gallosay
3  // * C#7 - 2nd Programming in C# II
4  // * 11/29/2020
5  // * Count to one recursion
6  // * Activity 3
7  // * References:
8  //
9
10 //=====
11 // Start of the Main Method
12 //=====
13
14 // Declare and Initialize
15
16 using System.Web;
17
18 int choice = 0, result = 0;
19 string input = "";
20
21 // Prompt the user for a number
22 Console.WriteLine("Enter a positive number: ");
23 // Get the users input
24 input = Console.ReadLine();
25
26 // See if the user entered valid input
27 while (!int.TryParse(input, out choice) && choice > 0)
28 {
29     // Display error
30     Console.ForegroundColor = ConsoleColor.Red;
31     Console.WriteLine("Invalid number");
32     Console.ResetColor();
33
34     // Re-prompt the user for a number
35     Console.WriteLine("Enter a positive number: ");
36
37     // Get the user input
38     input = Console.ReadLine();
39 }
40
41 // Call the CountToOne
42 result = Utility.CountToOne(choice);
43 Console.WriteLine($"The end number is {result}");
44
45 //=====
46 // End of the Main Method
47 //=====
```

Figure 1-7: Utility class and the count to one method.

PART 1 CHALLENGES

Screen Shots

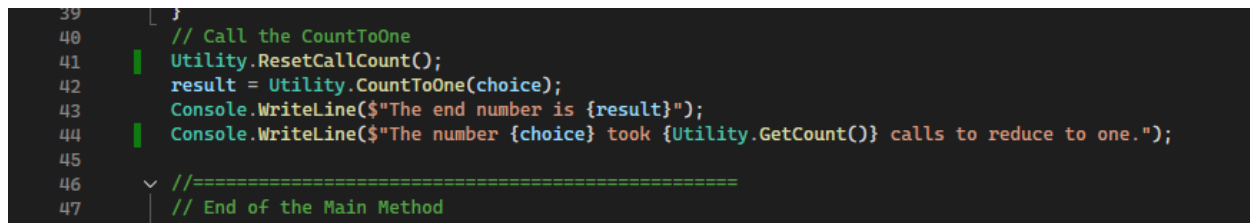


```
Microsoft Visual Studio Debug Console
The number is odd. Add 1
The current number is 72
The number is even. Divide by 2
The current number is 36
The number is even. Divide by 2
The current number is 18
The number is even. Divide by 2
The current number is 9
The number is odd. Add 1
The current number is 10
The number is even. Divide by 2
The current number is 5
The number is odd. Add 1
The current number is 6
The number is even. Divide by 2
The current number is 3
The number is odd. Add 1
The current number is 4
The number is even. Divide by 2
The current number is 2
The number is even. Divide by 2
The current number is 1
The end number is 1
The number 35972 took 27 calls to reduce to one.

C:\Users\WarWagon\OneDrive\School\Classes\CST-250\repo\Activity3\CountToOneRecursion\CountToOneRecursion\bin\Debug\net9.0\CountToOneRecursion.exe (process 35260) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops.
Press any key to close this window . . .
```

Figure 1-8: Task 1 challenge successful program completion.

Code



```
39 [
40 // Call the CountToOne
41 Utility.ResetCallCount();
42 result = Utility.CountToOne(choice);
43 Console.WriteLine($"The end number is {result}");
44 Console.WriteLine($"The number {choice} took {Utility.GetCount()} calls to reduce to one.");
45
46 //=====
47 // End of the Main Method
```

Figure 1-9: Changes to the main method for task 1 challenge.

```

Program.cs > X
CountToOneRecursion - Utility - callCount
51 //=====
52 // Start of the Utility class
53 //=====
54
55 // reference
56 static class Utility
57 {
58     static int callCount = 0;
59
60     /// <summary>
61     /// Count to one using recursion
62     /// </summary>
63     /// <param name="num">num</param>
64     /// <returns>/>returns
65     // reference
66     internal static int CountToOne(int num)
67     {
68         // increment call counter
69         callCount++;
70         // Print out the current number
71         Console.WriteLine($"The current number is {num}");
72         // Check if the number is 1: Base Case
73         if (num == 1)
74         {
75             return 1;
76         }
77         else
78         {
79             // Check if the number is even
80             if ((num % 2) == 0)
81             {
82                 Console.WriteLine("The number is even. Divide by 2");
83                 // Divide the number by 2 and call the function (recursion)
84                 return CountToOne(num / 2);
85             }
86             else
87             {
88                 Console.WriteLine("The number is odd. Add 1");
89                 // Add 1 and call the function (recursion)
90                 return CountToOne(num + 1);
91             }
92         }
93     }
94
95     // reference
96     internal static int GetCount()
97     {
98         return callCount;
99     }
100
101     // reference
102     internal static void ResetCallCount()
103     {
104         callCount = 0;
105     }
106 }

```

```
Microsoft Visual Studio Debug Console
Enter a positive number: 9
The current number is 9
The number is odd. Minus 1
The current number is 8
The number is even. Divide by 2
The current number is 4
The number is even. Divide by 2
The current number is 2
The number is even. Divide by 2
The current number is 1
The end number is 1
The number 9 took 5 calls to reduce to one.

C:\Users\WarWagon\OneDrive\School\Classes\CST-250\repo\Activity3\CountToOneRecursion\CountToOneRecursion\bin\Debug\net9.0\CountToOneRecursion.exe (process 44752) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops.
Press any key to close this window . . .
```

Figure 1-12: Changing to “minus one” runs successfully but the number of calls is reduced. In previous runs entering “9” would make eight calls using the plus one odd number adjustments. Using minus one does not adversely affect the programs operation but it reduced those eight calls to five. With larger numbers (35,972) this variation is less significant at ~ 25% but here in cases of low numbers it is an almost ~38% reduction in call overhead.

```
Microsoft Visual Studio Debug Console
The number is even. Divide by 2
The current number is 118
The number is even. Divide by 2
The current number is 59
The number is odd. Minus 1
The current number is 58
The number is even. Divide by 2
The current number is 29
The number is odd. Minus 1
The current number is 28
The number is even. Divide by 2
The current number is 14
The number is even. Divide by 2
The current number is 7
The number is odd. Minus 1
The current number is 6
The number is even. Divide by 2
The current number is 3
The number is odd. Minus 1
The current number is 2
The number is even. Divide by 2
The current number is 1
The end number is 1
The number 124556632 took 40 calls to reduce to one.

C:\Users\WarWagon\OneDrive\School\Classes\CST-250\repo\Activity3\CountToOneRecursion\CountToOneRecursion\bin\Debug\net9.0\CountToOneRecursion.exe (process 48552) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops.
Press any key to close this window . . .
```

Figure 1-13: Strangely, in the cases of 124,556,632 the number of calls using minus one increased by one compared to using plus one.

One thing that Figure 1-12 and 1-13 clearly demonstrate is don't be afraid to try different methods of adjusting outliers in your recursion loop. A simple thing like flipping a plus sign to minus had a massive effect on some of the lower number's computational efficiency.

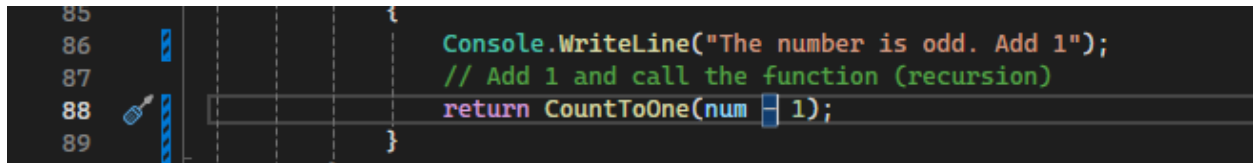


Figure 1-14: The only value changed to go from crashing to a reduction in calls was the number at the tail end of the return call to CountToOne

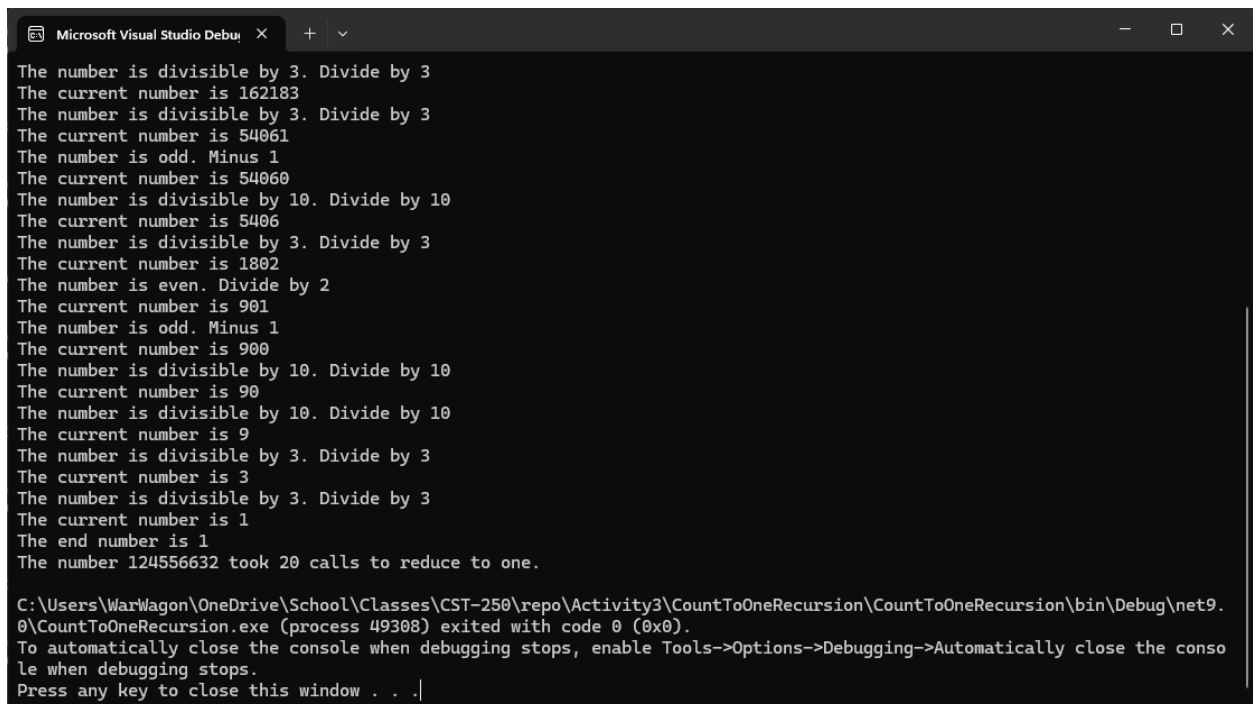


Figure 1-15: Extra Divisions added at mod 10, 5, 4, and 3 reduced the method calls to 20 for task 2 in the challenges list.


```

Microsoft Visual Studio Debug Console
The number is even. Divide by 2
The current number is -59
The number is odd. Shifting 1 number closer to zero
The current number is -58
The number is even. Divide by 2
The current number is -29
The number is odd. Shifting 1 number closer to zero
The current number is -28
The number is even. Divide by 2
The current number is -14
The number is even. Divide by 2
The current number is -7
The number is odd. Shifting 1 number closer to zero
The current number is -6
The number is even. Divide by 2
The current number is -3
The number is odd. Shifting 1 number closer to zero
The current number is -2
The number is even. Divide by 2
The current number is -1
The number is odd. Shifting 1 number closer to zero
The current number is 0
The end number is 0
The number -124556632 took 41 calls to reduce to one.

C:\Users\WarWagon\OneDrive\School\Classes\CST-250\repo\Activity3\CountToOneRecursion\CountToOneRecursion\bin\Debug\net9.0\CountToOneRecursion.exe (process 22844) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops.
Press any key to close this window . . .|

```

Figure 1-18: Running with negative number and zero tolerance.

```

Program.cs
CountToOneRecursion
//=====
// Start of the Utility class
//=====
// references
static class Utility
{
    // Declare and Initialize
    static int callCount = 0;
    static int OddNumberShiftValue = 0;

    // Summary
    // Count to one using recursion
    // Summary
    // param name="num" />param
    // returns=</returns>
    // references
    internal static int CountToOne(int num)
    {
        // increment call counter
        callCount++;
        // Print out the current number
        Console.WriteLine($"The current number is {num}");

        // toggle odd number modifier (shift value) between plus one and minus one depending on the value of num
        if (num > 0)
        {
            OddNumberShiftValue = -1;
        }
        else
        {
            OddNumberShiftValue = 1;
        }

        // Check if the number is 0: Base Case
        if (num == 0)
        {
            return 0;
        }
        else
        {
            // Check if the number is even
            if ((num % 2) == 0)
            {
                Console.WriteLine("The number is even. Divide by 2");
                // Divide the number by 2 and call the function (recursion)
                return CountToOne(num / 2);
            }
            else
            {
                Console.WriteLine("The number is odd. Shifting 1 number closer to zero");
                // Add 1 and call the function (recursion)
                return CountToOne(num + OddNumberShiftValue);
            }
        }
    }
}

```

Figure 1-19: Code changes to count to one for negative numbers

```

25
26 // See if the user entered valid input
27 while (!int.TryParse(input, out choice))
28 {
29     // Display error

```

Figure 1-20: Slightly altered error checking in main method to allow negative numbers

```

/*
* Chad Galloway
* CST - 250 Programming in C# II
* 11/02/2025
* Count to One
* Activity 2 Part 3i
* References:
*/

```

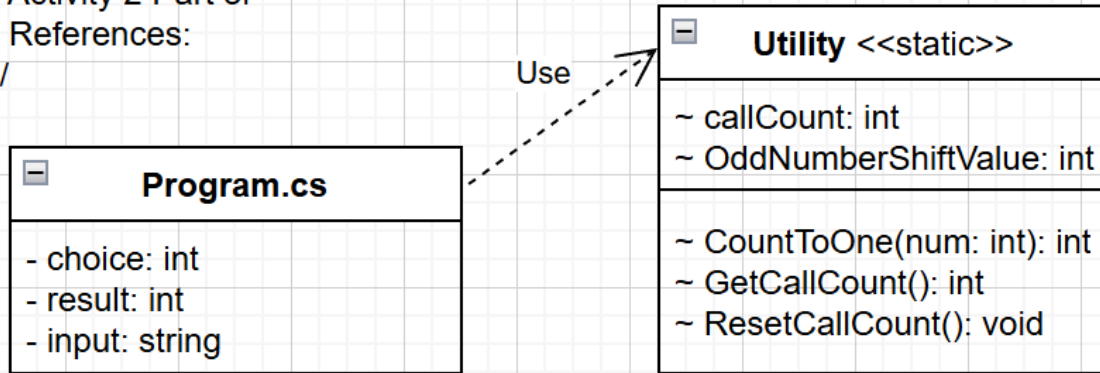


Figure 1-21: Final UML for the count to one console application

PART 2

Flow Chart

/*

* Chad Galloway

* CST - 250 Programming in C# II

* 11/02/2025

* Factorial

* Activity 3 Part 2

* References:

*/

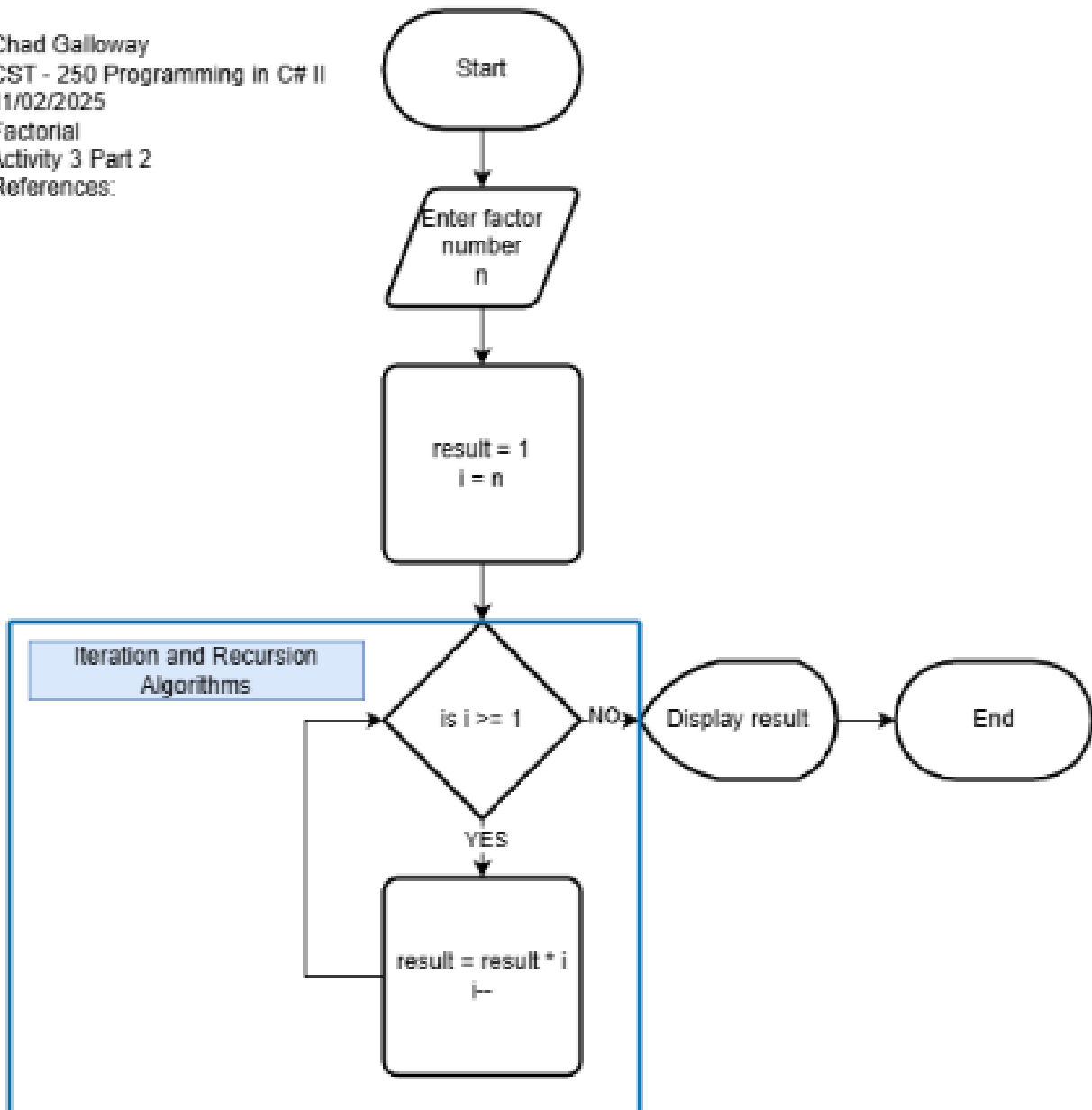


Figure 2-1: Flow chart of Activity 3 Part 2 Factorial recursion

UML Class Diagram

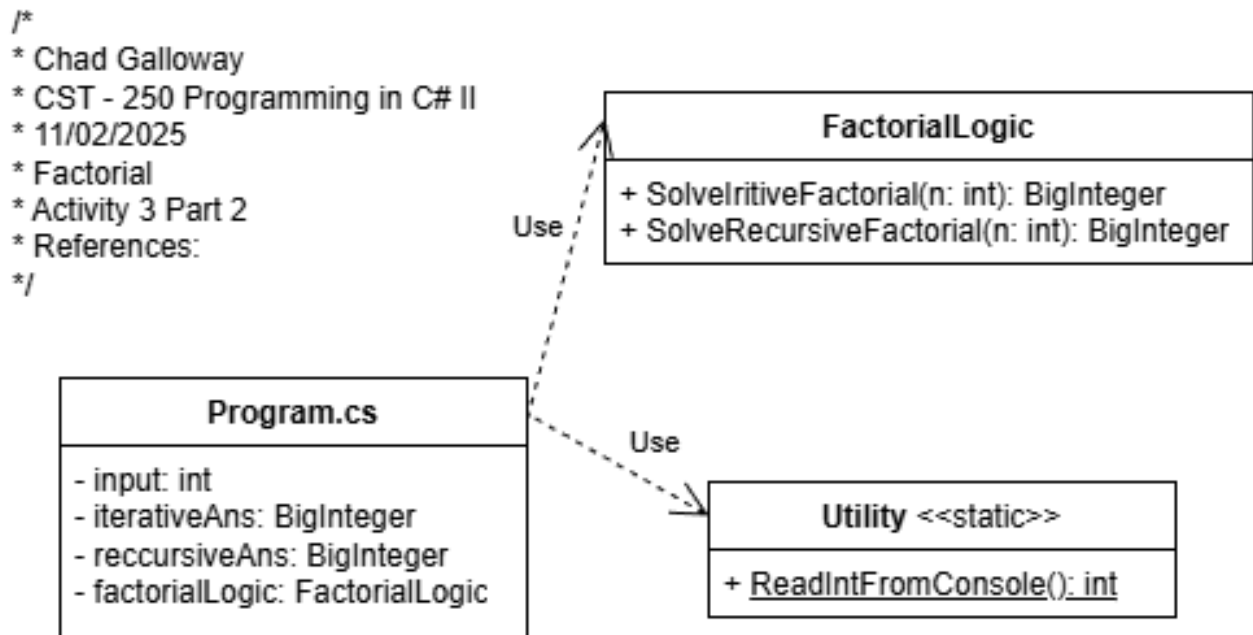


Figure 2-2: UML Class Diagram of Activity 3 Part 2 Factorial recursion

Screen Shots

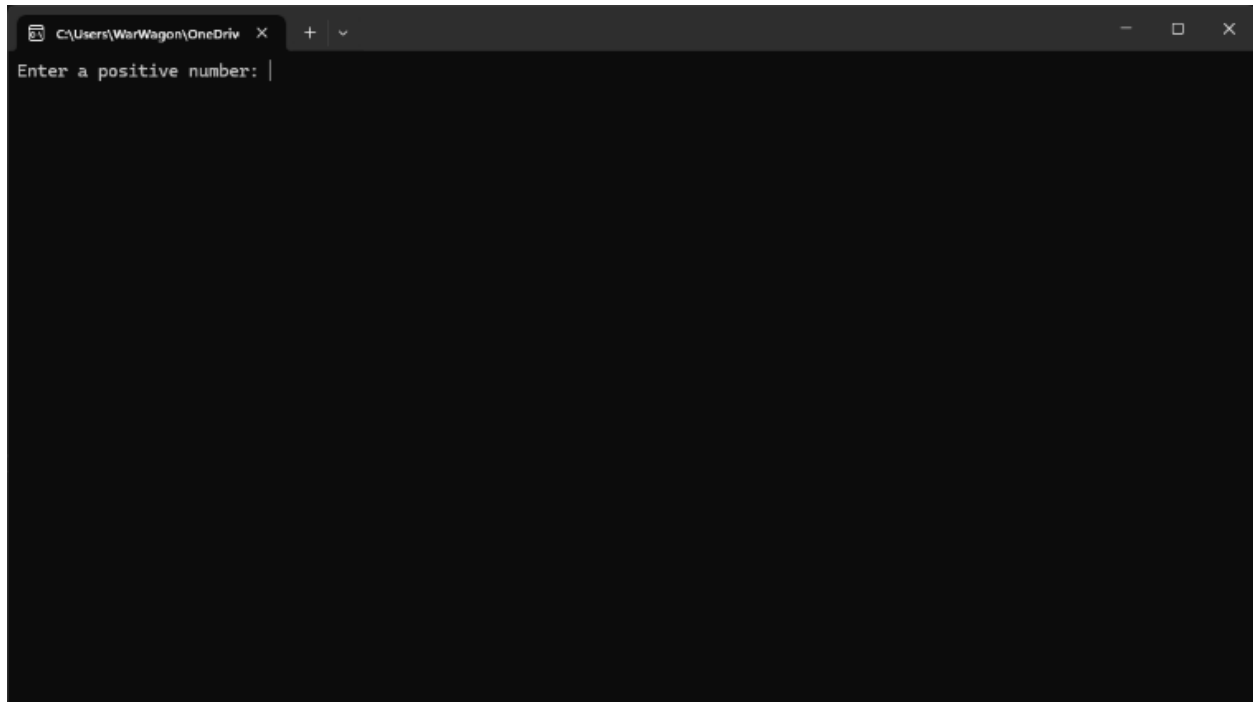
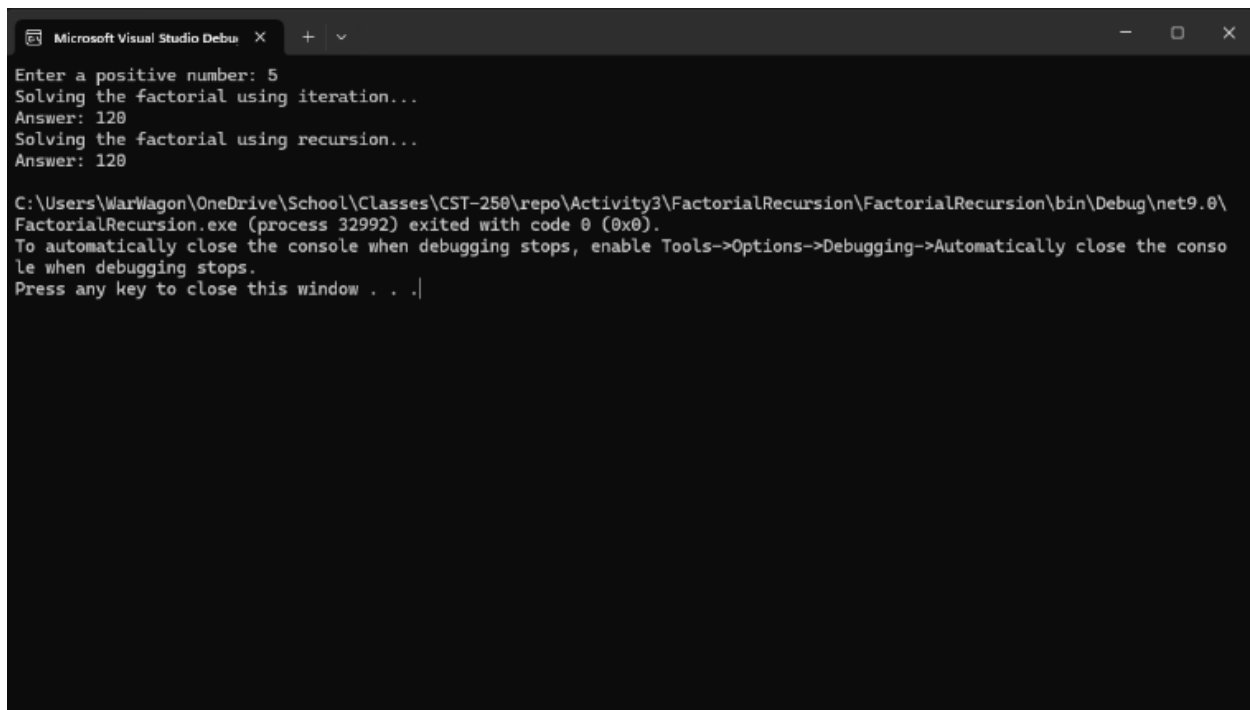


Figure 2-3: Part 2 App Running initial load state

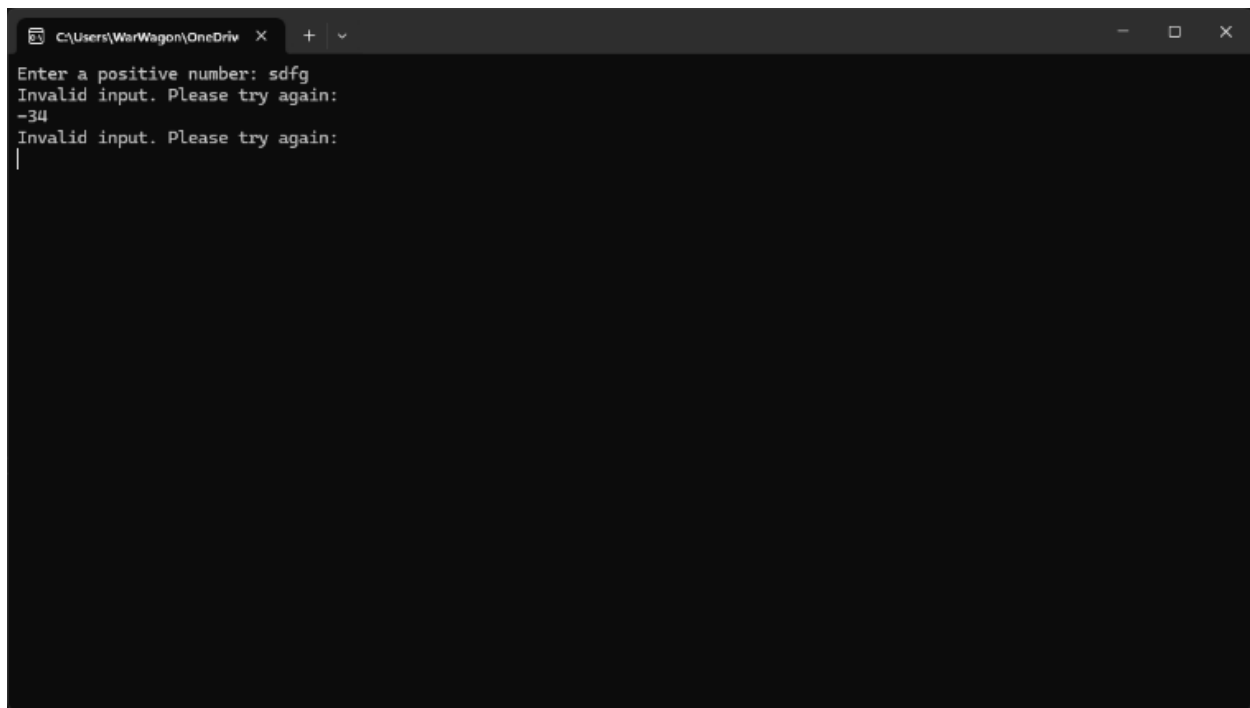


A screenshot of a Microsoft Visual Studio Debug Console window. The window title is "Microsoft Visual Studio Debug Console". The text inside the console shows the following sequence of events: a prompt "Enter a positive number:" followed by the input "5"; the text "Solving the factorial using iteration..." followed by "Answer: 120"; and then "Solving the factorial using recursion..." followed by "Answer: 120". At the bottom, there is a message: "C:\Users\WarWagon\OneDrive\School\Classes\CST-250\repo\Activity3\FactorialRecursion\FactorialRecursion\bin\Debug\net9.0\FactorialRecursion.exe (process 32992) exited with code 0 (0x0). To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops. Press any key to close this window . . .|".

```
Enter a positive number: 5
Solving the factorial using iteration...
Answer: 120
Solving the factorial using recursion...
Answer: 120

C:\Users\WarWagon\OneDrive\School\Classes\CST-250\repo\Activity3\FactorialRecursion\FactorialRecursion\bin\Debug\net9.0\FactorialRecursion.exe (process 32992) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops.
Press any key to close this window . . .|
```

Figure 2-4: Screenshot of successful output Factoring 5



A screenshot of a Microsoft Visual Studio Debug Console window. The window title is "C:\Users\WarWagon\OneDrive\School\Classes\CST-250\repo\Activity3\FactorialRecursion\FactorialRecursion\bin\Debug\net9.0\FactorialRecursion.exe". The text inside the console shows the following sequence of events: a prompt "Enter a positive number:" followed by the input "sdfg"; the text "Invalid input. Please try again:" followed by the input "-34"; and then "Invalid input. Please try again:" followed by a blank line. The cursor is at the end of the blank line.

```
Enter a positive number: sdfg
Invalid input. Please try again:
-34
Invalid input. Please try again:
|
```

Figure 2-5: Screenshot of Error Handling non integer numbers

Code

```
Programs - X
FactorialRecursion - %Utility - %ReadIntFromConsole()
1  /*
2   * Chad Galloway
3   * CST - 258 Programming in C# II
4   * 11/09/2020
5   * Factorial recursion
6   * Activity 3 Part 2
7   * References:
8   */
9
10 //-----
11 // Start of the Main Method
12 //-----
13
14 // Declare and Initialize
15 using FactorialRecursion.Services.BusinessLogicLayer;
16 using System.Numerics;
17
18 FactorialLogic factorialLogic = new FactorialLogic();
19 int input = 0;
20 BigInteger iterativeAns = 0, recursiveAns = 0;
21
22 // Prompt the user
23 Console.WriteLine("Enter a positive number: ");
24
25 // Get the users input
26 input = Utility.ReadIntFromConsole();
27
28 // Solve the factorial using iteration
29 Console.WriteLine("Solving the factorial using iteration...");
30 iterativeAns = factorialLogic.SolveIterativeFactorial(input);
31 Console.WriteLine($"Answer: {iterativeAns}");
32
33 // Solve the factorial using recursion
34 Console.WriteLine("Solving the factorial using recursion...");
35 recursiveAns = factorialLogic.SolveRecursiveFactorial(input);
36 Console.WriteLine($"Answer: {recursiveAns}");
37
38 //-----
39 // End of the Main Method
40 //-----
41
```

Figure 2-6: Code Citations and the main method.

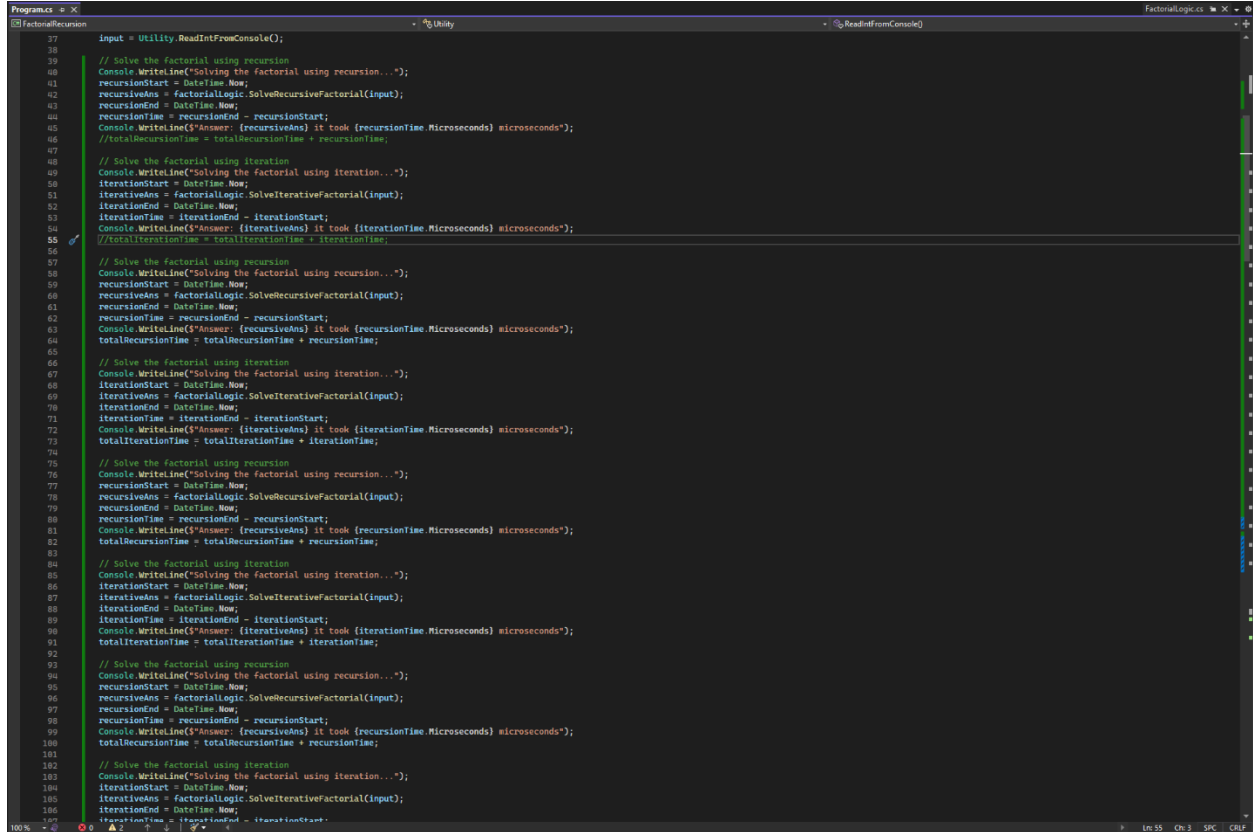
```
Programs - X
FactorialRecursion - %Utility - %ReadIntFromConsole()
43 //-----
44 // Start of the Utility class
45 //-----
46
47 /// <summary>
48 /// Read Integer input from the console
49 /// </summary>
50 /// <returns>-->returns</returns>
51 //-----
52 static class Utility
53 {
54     //reference
55     internal static int ReadIntFromConsole()
56     {
57         // Declare and Initialize
58         string input = "";
59         int number = -1;
60
61         // Get the current input
62         input = Console.ReadLine();
63
64         // Check if the input is valid
65         while ( !int.TryParse(input, out number) && number > 0 )
66         {
67             // Show the user an error message and prompt them again
68             Console.WriteLine("Invalid input. Please try again: ");
69
70             // Get the new input
71             input = Console.ReadLine();
72         }
73
74         // Return the resulting integer from the user
75         return number;
76     }
77
78 //-----
79 // End of the Utility class
80 //-----
81
82
```

Figure 2-7: Utility class and the read int from console method.


```
FactorialLogic.cs
1  // Chad Galloway
2  // CST - 200 Programming in C# II
3  // 11/09/2020
4  // Factorial recursion
5  // Activity 3 Part 2
6  // References:
7
8
9
10 using System.Numerics;
11
12 namespace FactorialRecursion.Services.BusinessLogicLayer
13 {
14     [reference]
15     internal class FactorialLogic
16     {
17         /// <summary>
18         /// Solve the factorial problem using iteration
19         /// </summary>
20         /// <param name="factorial"></param>
21         /// <returns></returns>
22         internal BigInteger SolveIterativeFactorial(int factorial)
23         {
24             // Declare and Initialize
25             BigInteger result = 1;
26
27             // For loop to solve the factorial
28             for (int i = factorial; i >= 1; i--)
29             {
30                 // Multiply the result by i
31                 result *= i;
32             }
33
34             // Return the result
35             return result;
36         }
37
38         /// <summary>
39         /// Solve the factorial problem using recursion
40         /// </summary>
41         /// <param name="factorial"></param>
42         /// <returns></returns>
43         internal BigInteger SolveRecursiveFactorial(int factorial)
44         {
45             // Base case: factorial is 0 or 1
46             if (factorial == 0 || factorial == 1)
47             {
48                 return 1;
49             }
50
51             // Perform the recursion
52             return factorial * SolveRecursiveFactorial(factorial - 1);
53         }
54     }
55 }
```

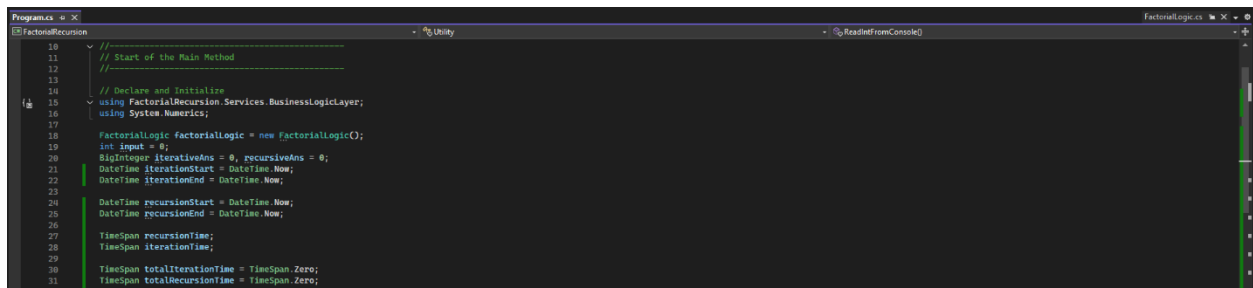
Figure 2-8: Factorial logic class with the two methods for iteration and recursion present

Code



```
17 input = Utility.ReadIntFromConsole();
18
19 // Solve the factorial using recursion
20 Console.WriteLine("Solving the factorial using recursion...");
21 recursionStart = DateTime.Now;
22 recursiveAns = factorialLogic.SolveRecursiveFactorial(input);
23 recursionEnd = DateTime.Now;
24 recursionTime = recursionEnd - recursionStart;
25 Console.WriteLine($"Answer: {recursiveAns} it took {recursionTime.Microseconds} microseconds");
26 //totalRecursionTime = totalRecursionTime + recursionTime;
27
28 // Solve the factorial using iteration
29 Console.WriteLine("Solving the factorial using iteration...");
30 iterationStart = DateTime.Now;
31 iterativeAns = factorialLogic.SolveIterativeFactorial(input);
32 iterationEnd = DateTime.Now;
33 iterationTime = iterationEnd - iterationStart;
34 Console.WriteLine($"Answer: {iterativeAns} it took {iterationTime.Microseconds} microseconds");
35 //totalIterationTime = totalIterationTime + iterationTime;
36
37 // Solve the factorial using recursion
38 Console.WriteLine("Solving the factorial using recursion...");
39 recursionStart = DateTime.Now;
40 recursiveAns = factorialLogic.SolveRecursiveFactorial(input);
41 recursionEnd = DateTime.Now;
42 recursionTime = recursionEnd - recursionStart;
43 Console.WriteLine($"Answer: {recursiveAns} it took {recursionTime.Microseconds} microseconds");
44 totalRecursionTime = totalRecursionTime + recursionTime;
45
46 // Solve the factorial using iteration
47 Console.WriteLine("Solving the factorial using iteration...");
48 iterationStart = DateTime.Now;
49 iterativeAns = factorialLogic.SolveIterativeFactorial(input);
50 iterationEnd = DateTime.Now;
51 iterationTime = iterationEnd - iterationStart;
52 Console.WriteLine($"Answer: {iterativeAns} it took {iterationTime.Microseconds} microseconds");
53 totalIterationTime = totalIterationTime + iterationTime;
54
55 // Solve the factorial using recursion
56 Console.WriteLine("Solving the factorial using recursion...");
57 recursionStart = DateTime.Now;
58 recursiveAns = factorialLogic.SolveRecursiveFactorial(input);
59 recursionEnd = DateTime.Now;
60 recursionTime = recursionEnd - recursionStart;
61 Console.WriteLine($"Answer: {recursiveAns} it took {recursionTime.Microseconds} microseconds");
62 totalRecursionTime = totalRecursionTime + recursionTime;
63
64 // Solve the factorial using iteration
65 Console.WriteLine("Solving the factorial using iteration...");
66 iterationStart = DateTime.Now;
67 iterativeAns = factorialLogic.SolveIterativeFactorial(input);
68 iterationEnd = DateTime.Now;
69 iterationTime = iterationEnd - iterationStart;
70 Console.WriteLine($"Answer: {iterativeAns} it took {iterationTime.Microseconds} microseconds");
71 totalIterationTime = totalIterationTime + iterationTime;
72
73 // Solve the factorial using recursion
74 Console.WriteLine("Solving the factorial using recursion...");
75 recursionStart = DateTime.Now;
76 recursiveAns = factorialLogic.SolveRecursiveFactorial(input);
77 recursionEnd = DateTime.Now;
78 recursionTime = recursionEnd - recursionStart;
79 Console.WriteLine($"Answer: {recursiveAns} it took {recursionTime.Microseconds} microseconds");
80 totalRecursionTime = totalRecursionTime + recursionTime;
81
82 // Solve the factorial using iteration
83 Console.WriteLine("Solving the factorial using iteration...");
84 iterationStart = DateTime.Now;
85 iterativeAns = factorialLogic.SolveIterativeFactorial(input);
86 iterationEnd = DateTime.Now;
87 iterationTime = iterationEnd - iterationStart;
88 Console.WriteLine($"Answer: {iterativeAns} it took {iterationTime.Microseconds} microseconds");
89 totalIterationTime = totalIterationTime + iterationTime;
90
91 // Solve the factorial using recursion
92 Console.WriteLine("Solving the factorial using recursion...");
93 recursionStart = DateTime.Now;
94 recursiveAns = factorialLogic.SolveRecursiveFactorial(input);
95 recursionEnd = DateTime.Now;
96 recursionTime = recursionEnd - recursionStart;
97 Console.WriteLine($"Answer: {recursiveAns} it took {recursionTime.Microseconds} microseconds");
98 totalRecursionTime = totalRecursionTime + recursionTime;
99
100 // Solve the factorial using iteration
101 Console.WriteLine("Solving the factorial using iteration...");
102 iterationStart = DateTime.Now;
103 iterativeAns = factorialLogic.SolveIterativeFactorial(input);
104 iterationEnd = DateTime.Now;
105 iterationTime = iterationEnd - iterationStart;
106 Console.WriteLine($"Answer: {iterativeAns} it took {iterationTime.Microseconds} microseconds");
107 totalIterationTime = totalIterationTime + iterationTime;
```

Figure 2-10: Code for the increased number of method calls. I even tried a structure where the calls happened in a different order and that just widened the gap between the two different structure types. So based off of all my tests at 6000! Iteration has a clear edge of about 30% efficiency.



```
10 // Start of the Main Method
11 //
12 //
13 //
14 // Declare and Initialize
15 using FactorialRecursion.Services.BusinessLogicLayer;
16 using System.Numerics;
17
18 FactorialLogic factorialLogic = new FactorialLogic();
19 int input = 0;
20 BigInteger iterativeAns = 0, recursiveAns = 0;
21 DateTime iterationStart = DateTime.Now;
22 DateTime iterationEnd = DateTime.Now;
23
24 DateTime recursionStart = DateTime.Now;
25 DateTime recursionEnd = DateTime.Now;
26
27 TimeSpan recursionTime;
28 TimeSpan iterationTime;
29
30 TimeSpan totalIterationTime = TimeSpan.Zero;
31 TimeSpan totalRecursionTime = TimeSpan.Zero;
32
```

Figure 2-11: The code for the tracking variables I created.

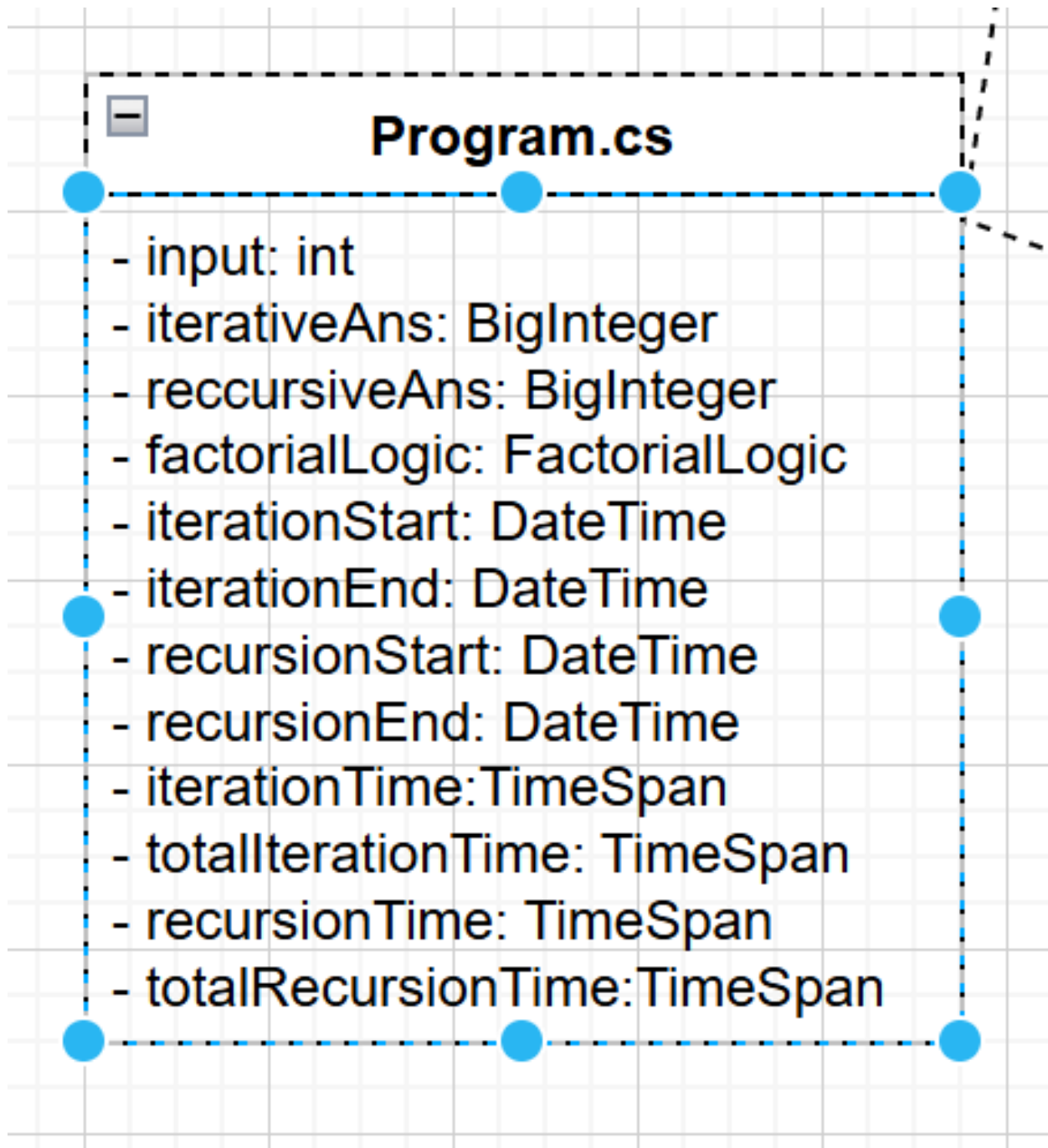


Figure 2-12: Updated UML for my timing tests

PART 3

Flow Chart

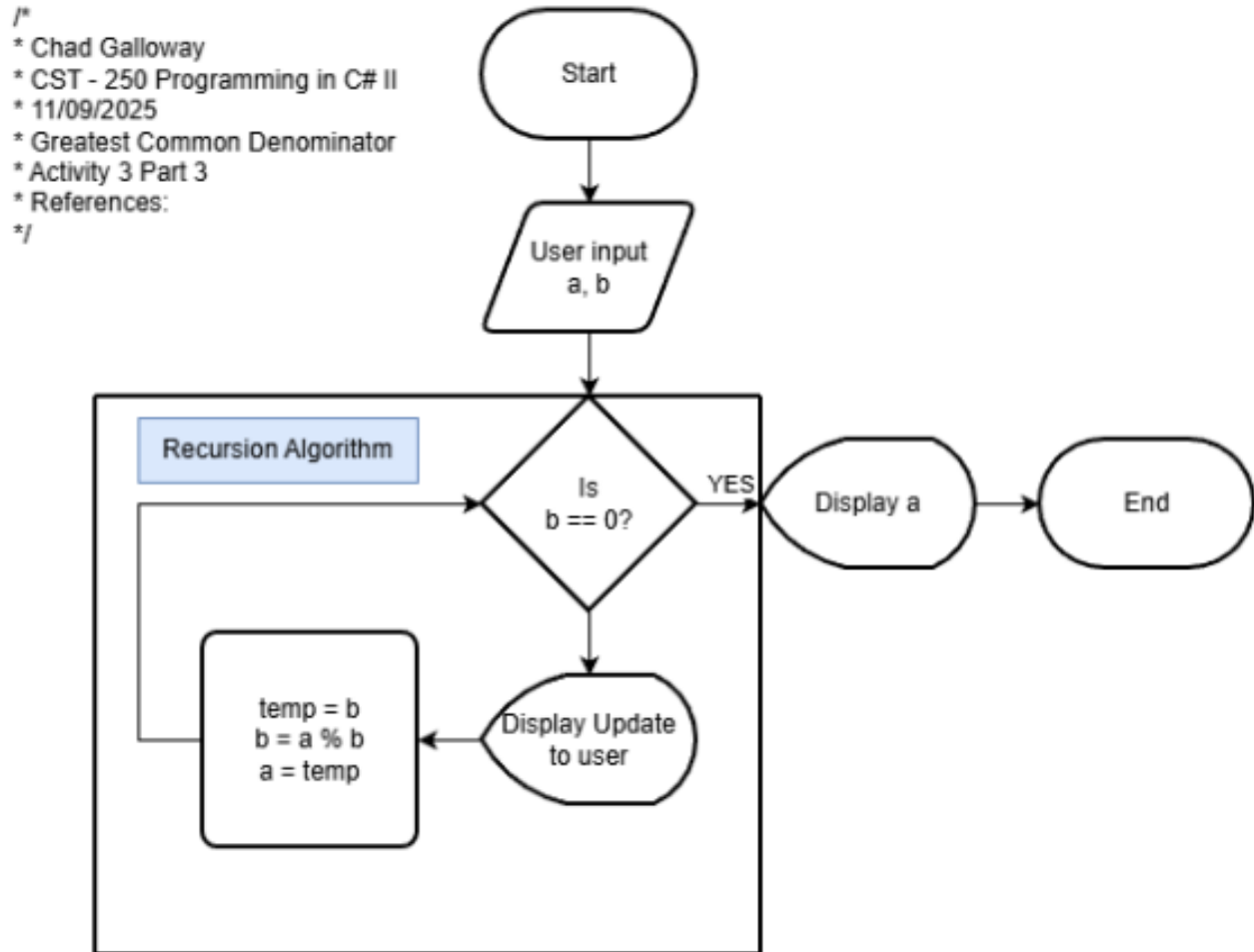


Figure 3-1: Flow chart of Activity 3 Part 3 greatest common divisor

UML Class Diagram

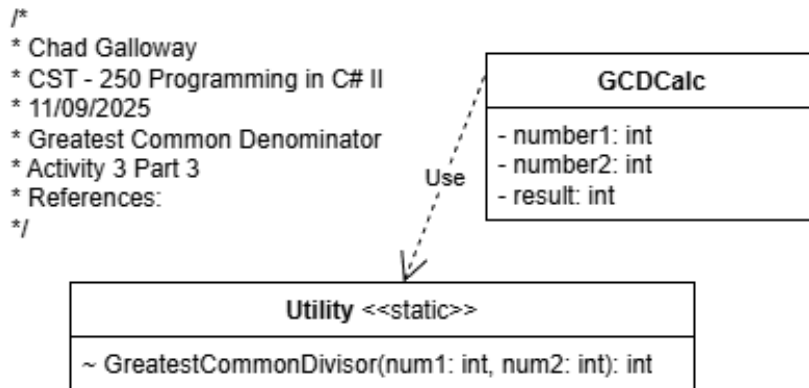


Figure 3-2: UML Class Diagram of Activity 3 Part 3 greatest common divisor

Screen Shots

The screenshot shows a console window with the following text:

```
Microsoft Visual Studio Debu x + v
Not yet. The remainder of 440 and 80 is 40.
Not yet. The remainder of 80 and 40 is 0.
The GCD of 440 and 80 is 40

C:\Users\WarWagon\OneDrive\School\Classes\CST-250\repo\Activity3\GreatestCommonDivisorRecursion\GreatestCommonDivisorRecursion\bin\Debug\net9.0\GreatestCommonDivisorRecursion.exe (process 45728) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops.
Press any key to close this window . . .|
```

Figure 3-3: Screenshot of successful output

Code

```
Programs * X
GreatestCommonDivisorRecursion - *g Utility - |GreatestCommonDivisor(int num1, int num2)
1  /*
2   * Chad Galloway
3   * CST - 250 Programming in C# II
4   * 11/09/2020
5   * Greatest Common Divisor Recursion
6   * Activity 3 Part 3
7   * References:
8   */
9
10 //-----
11 // Start of the Main Method
12 //-----
13
14 // Declare and Initialize
15 int number1 = 408, number2 = 80, result = 0;
16
17 // Call the GCD
18 result = Utility.GreatestCommonDivisor(number1, number2);
19
20 // Print the result to the user
21 Console.WriteLine($"The GCD of {number1} and {number2} is {result}");
22
23 //-----
24 // End of the Main Method
25 //-----
26
27
28
29 //-----
30 // Start of the Utility Class
31 //-----
32
33
34 // reference
35 public class Utility
36 {
37     // reference
38     internal static int GreatestCommonDivisor(int num1, int num2)
39     {
40         // Declare and Initialize
41         int remainder = 0;
42
43         // Base Case: num2 = 0
44         if (num1 == 0 || num2 == 0)
45         {
46             // Return the greatest common divisor
47             return num1;
48         }
49         else
50         {
51             // Get the remainder of dividing num1 and num2
52             remainder = num1 % num2;
53             // Print an update to the user
54             Console.WriteLine($"Not yet. The remainder of {num1} and {num2} is {remainder}.");
55             // Call the recursive function of the second number and the remainder
56             return GreatestCommonDivisor(num2, remainder);
57         }
58     }
59 }
60
61 //-----
62 // End of the Utility Class
63 //-----
```

Figure 3-4: Code Citations and the main method.

PART 3 CHALLENGES

Screen Shots

```
Microsoft Visual Studio Debug Console
How many numbers do you want to use?: 5
Please enter each number:
Number 1: 440
Number 2: 80
Number 3: 160
Number 4: 320
Number 5: 4000
Starting to process the solution using iteration for depth and recursion for solution
Not yet. The remainder of 80 and 440 is 80.
Not yet. The remainder of 440 and 80 is 40.
Not yet. The remainder of 80 and 40 is 0.
Not yet. The remainder of 160 and 40 is 0.
Not yet. The remainder of 320 and 40 is 0.
Not yet. The remainder of 4000 and 40 is 0.
Using iteration for depth and recursion for solution is 40, it took 0.3276
Starting to process the solution using iteration for depth and iteration for solution
Not yet. The remainder of 80 and 440 is 80.
Not yet. The remainder of 80 and 440 is 40.
Not yet. The remainder of 80 and 440 is 0.
Not yet. The remainder of 160 and 40 is 0.
Not yet. The remainder of 320 and 40 is 0.
Not yet. The remainder of 4000 and 40 is 0.
Using iteration for depth and iteration for solution is 40, it took 0.2219
Starting to process the solution using iteration for depth and counting for solution
The first list has 10 entries.
The second list has 16 entries.
The first list has 12 entries.
The second list has 8 entries.
The first list has 14 entries.
The second list has 8 entries.
The first list has 24 entries.
The second list has 8 entries.
Using iteration for depth and counting for solution is 40, it took 5.1697
Starting to process the solution using recursion for depth and recursion for solution
Still more numbers. Calling the next number pair 440 and 80.
Still more numbers. Calling the next number pair 80 and 160.
Still more numbers. Calling the next number pair 160 and 320.
Still more numbers. Calling the next number pair 320 and 4000.
4000 is the last number in the array, collapsing the solution back down.
Not yet. The remainder of 320 and 4000 is 320.
Not yet. The remainder of 4000 and 320 is 160.
Not yet. The remainder of 320 and 160 is 0.
Not yet. The remainder of 160 and 160 is 0.
Not yet. The remainder of 80 and 160 is 80.
Not yet. The remainder of 160 and 80 is 0.
Not yet. The remainder of 440 and 80 is 40.
Not yet. The remainder of 80 and 40 is 0.
Using recursion for depth and recursion for solution is 40, it took 0.4712
Starting to process the solution using recursion for depth and iteration for solution
Still more numbers. Calling the next number pair 440 and 80.
Still more numbers. Calling the next number pair 80 and 160.
Still more numbers. Calling the next number pair 160 and 320.
Still more numbers. Calling the next number pair 320 and 4000.
4000 is the last number in the array, collapsing the solution back down.
Not yet. The remainder of 320 and 4000 is 320.
Not yet. The remainder of 320 and 4000 is 160.
Not yet. The remainder of 320 and 4000 is 0.
Not yet. The remainder of 160 and 160 is 0.
Not yet. The remainder of 80 and 160 is 80.
Not yet. The remainder of 80 and 160 is 0.
Not yet. The remainder of 440 and 80 is 40.
Not yet. The remainder of 440 and 80 is 0.
Using recursion for depth and iteration for solution is 40, it took 0.3412
Starting to process the solution using recursion for depth and counting for solution
Still more numbers. Calling the next number pair 440 and 80.
```

Figure 3-5 Screen shot of the user entry on the challenges. Completing number one of the challenges


```
Microsoft Visual Studio Debug Console
Still more numbers. Calling the next number pair 440 and 80.
Still more numbers. Calling the next number pair 80 and 160.
Still more numbers. Calling the next number pair 160 and 320.
Still more numbers. Calling the next number pair 320 and 4000.
4000 is the last number in the array, collapsing the solution back down.
Not yet. The remainder of 320 and 4000 is 320.
Not yet. The remainder of 4000 and 320 is 160.
Not yet. The remainder of 320 and 160 is 0.
Not yet. The remainder of 160 and 160 is 0.
Not yet. The remainder of 80 and 160 is 80.
Not yet. The remainder of 160 and 80 is 0.
Not yet. The remainder of 440 and 80 is 40.
Not yet. The remainder of 80 and 40 is 0.
Using recursion for depth and recursion for solution is 40, it took 0.2206
Starting to process the solution using recursion for depth and counting for solution
Still more numbers. Calling the next number pair 440 and 80.
Still more numbers. Calling the next number pair 80 and 160.
Still more numbers. Calling the next number pair 160 and 320.
Still more numbers. Calling the next number pair 320 and 4000.
4000 is the last number in the array, collapsing the solution back down.
The first list has 14 entries.
The second list has 24 entries.
The first list has 12 entries.
The second list has 12 entries.
The first list has 10 entries.
The second list has 12 entries.
The first list has 16 entries.
The second list has 10 entries.
Using recursion for depth and counting for solution is 40, it took 0.2091
Starting to process the solution using iteration for depth and iteration for solution
Not yet. The remainder of 80 and 440 is 80.
Not yet. The remainder of 80 and 440 is 40.
Not yet. The remainder of 80 and 440 is 0.
Not yet. The remainder of 160 and 40 is 0.
Not yet. The remainder of 320 and 40 is 0.
Not yet. The remainder of 4000 and 40 is 0.
Using iteration for depth and iteration for solution is 40, it took 0.0902
Starting to process the solution using iteration for depth and recursion for solution
Not yet. The remainder of 80 and 440 is 80.
Not yet. The remainder of 440 and 80 is 40.
Not yet. The remainder of 80 and 40 is 0.
Not yet. The remainder of 160 and 40 is 0.
Not yet. The remainder of 320 and 40 is 0.
Not yet. The remainder of 4000 and 40 is 0.
Using iteration for depth and recursion for solution is 40, it took 0.0908
Starting to process the solution using iteration for depth and counting for solution
The first list has 10 entries.
The second list has 16 entries.
The first list has 12 entries.
The second list has 8 entries.
The first list has 14 entries.
The second list has 8 entries.
The first list has 24 entries.
The second list has 8 entries.
Using iteration for depth and counting for solution is 40, it took 0.1321
You entered 5 numbers
They were: 440, 80, 160, 320, 4000,
The greatest common divisor for these numbers was 40
The time taken doing iteration depth and iteration solution was 0.0009287 seconds or 0.9287 milliseconds
The time taken doing iteration depth and recursion solution was 0.0011788 seconds or 1.1788 milliseconds
The time taken doing iteration depth and counting solution was 0.0014628 seconds or 1.4628 milliseconds
The time taken doing recursion depth and iteration solution was 0.002282 seconds or 2.282 milliseconds
The time taken doing recursion depth and recursion solution was 0.002057 seconds or 2.057 milliseconds
The time taken doing recursion depth and counting solution was 0.0021151 seconds or 2.1151 milliseconds
```

Figure 3-6 Screen shot of the successful completion of the challenges. Every different type of method call is color coded. It made back tracking when one type of operation had an unusually high time or even an incorrect result when I worked on making negative numbers and zero function properly. This screen shot depicts challenges 2-4

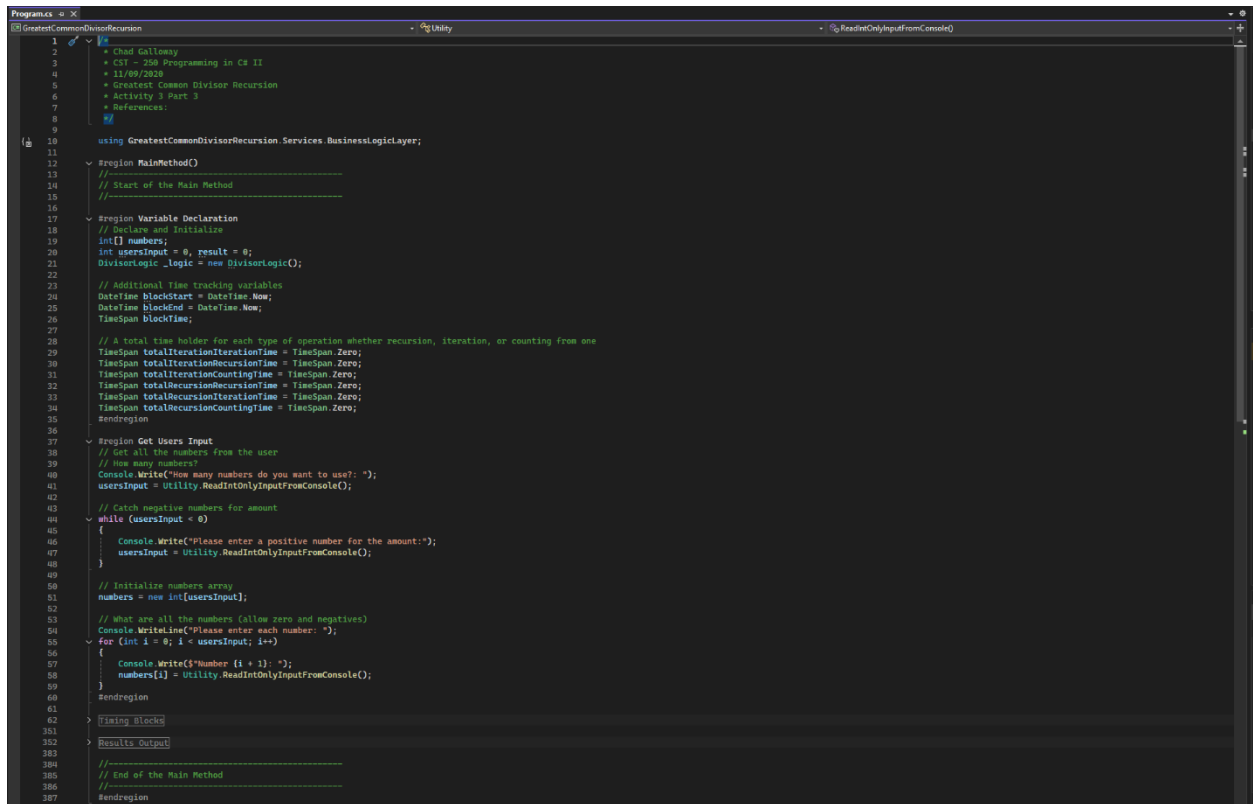
```
C:\Users\WarWagon\OneDrive x + v
How many numbers do you want to use?: -5
Please enter a positive number for the amount:6
Please enter each number:
Number 1: sfdg
Invalid input. Please try again:
-440
Number 2: 80
Number 3: 0
Number 4: 320
Number 5: -4000
Number 6: 3300
```

Figure 3-7 Screen shot of the error handling capabilities of the application.

```
Microsoft Visual Studio Debu x + v
Using iteration for depth and recursion for solution is 20, it took 0.1213
Starting to process the solution using iteration for depth and counting for solution
The first list has 6 entries.
The second list has 6 entries.
The first list has 12 entries.
The second list has 6 entries.
The first list has 12 entries.
The second list has 6 entries.
The first list has 20 entries.
The second list has 6 entries.
The first list has 20 entries.
The second list has 6 entries.
The first list has 10 entries.
The second list has 6 entries.
Using iteration for depth and counting for solution is 20, it took 0.1893
You entered 10 numbers
They were: 20, 0, -20, -200, 0, 200, 2000, 0, -2000, 80,
The greatest common divisor for these numbers was 20
The time taken doing iteration depth and iteration solution was 0.0009839 seconds or 0.9839 milliseconds
The time taken doing iteration depth and recursion solution was 0.0010422 seconds or 1.0422 milliseconds
The time taken doing iteration depth and counting solution was 0.0022252 seconds or 2.2252 milliseconds
The time taken doing recursion depth and iteration solution was 0.003285 seconds or 3.285 milliseconds
The time taken doing recursion depth and recursion solution was 0.0031061 seconds or 3.1061 milliseconds
The time taken doing recursion depth and counting solution was 0.0037193 seconds or 3.7193 milliseconds
C:\Users\WarWagon\OneDrive\School\Classes\CST-250\repo\Activity3\GreatestCommonDivisorRecursion\GreatestCommonDivisorRecursion\bin\Debug\net9.0\GreatestCommonDivisorRecursion.exe (process 17640) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops.
Press any key to close this window . . .
```

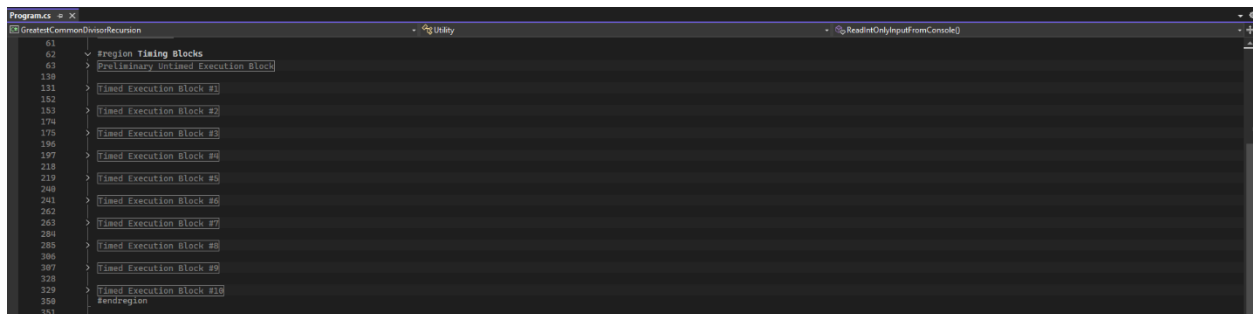
Figure 3-8 Screen shot of the zero and negative handling of the application. Figure 3-7 and 3-8 satisfy number 5 of the part three challenges.

Code



```
1  //
2  // * Chad Galloway
3  // * C# - 250 Programming in C# II
4  // * 11/09/2020
5  // * Greatest Common Divisor Recursion
6  // * Activity 3 Part 3
7  // * References:
8  //
9
10 using GreatestCommonDivisorRecursion.Services.BusinessLogicLayer;
11
12 #region MainMethod()
13 //-----
14 // Start of the Main Method
15 //-----
16
17 #region Variable Declaration
18 // Declare and Initialize
19 int[] numbers;
20 int usersInput = 0, result = 0;
21 DivisorLogic _logic = new DivisorLogic();
22
23 // Additional Time tracking variables
24 DateTime blockStart = DateTime.Now;
25 DateTime blockEnd = DateTime.Now;
26 TimeSpan blockTime;
27
28 // A total time holder for each type of operation whether recursion, iteration, or counting from one
29 TimeSpan totalIterationIterationTime = TimeSpan.Zero;
30 TimeSpan totalIterationRecursionTime = TimeSpan.Zero;
31 TimeSpan totalIterationCountingTime = TimeSpan.Zero;
32 TimeSpan totalRecursionRecursionTime = TimeSpan.Zero;
33 TimeSpan totalRecursionIterationTime = TimeSpan.Zero;
34 TimeSpan totalRecursionCountingTime = TimeSpan.Zero;
35 #endregion
36
37 #region Get Users Input
38 // Get all the numbers from the user
39 // How many numbers?
40 Console.WriteLine("How many numbers do you want to use?: ");
41 usersInput = Utility.ReadIntOnlyInputFromConsole();
42
43 // Catch negative numbers for amount
44 while (usersInput < 0)
45 {
46     Console.WriteLine("Please enter a positive number for the amount:");
47     usersInput = Utility.ReadIntOnlyInputFromConsole();
48 }
49
50 // Initialize numbers array
51 numbers = new int[usersInput];
52
53 // What are all the numbers (allow zero and negatives)
54 Console.WriteLine("Please enter each number: ");
55 for (int i = 0; i < usersInput; i++)
56 {
57     Console.WriteLine($"Number {i + 1}: ");
58     numbers[i] = Utility.ReadIntOnlyInputFromConsole();
59 }
60 #endregion
61
62 #region Timing Blocks
63 //-----
64 // Results Output
65 //-----
66 // End of the Main Method
67 //-----
68 #endregion
```

Figure 3-9: Code Citations and the main method.



```
61
62 #region Timing Blocks
63 > Preliminary Untimed Execution Block
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111 > Timed Execution Block #1
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152 > Timed Execution Block #2
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311 > Timed Execution Block #10
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
```

Figure 3-10: A collapsed region of all the different code blocks used to track time on multiple method calls

```
17 > Variable Declaration
36
37 > Get Users Input
61
62 < Region Timing Blocks
63 < Preliminary Untimed Execution Block
64 //-----
65 // Start Preliminary Untimed Execution Block
66 //-----
67
68 Console.ForegroundColor = ConsoleColor.Yellow;
69 Console.WriteLine("Starting to process the solution using iteration for depth and recursion for solution");
70 blockStart = DateTime.Now;
71 result = Logic.SolveRecursiveGCD(numbers);
72 blockEnd = DateTime.Now;
73 blockTime = blockEnd - blockStart;
74 // Print the result to the user
75 Console.WriteLine($"Using iteration for depth and recursion for solution is {result}, it took {blockTime.TotalMilliseconds}");
76
77 Console.ForegroundColor = ConsoleColor.Red;
78 Console.WriteLine("Starting to process the solution using iteration for depth and iteration for solution");
79 blockStart = DateTime.Now;
80 result = Logic.SolveIterativeGCD(numbers);
81 blockEnd = DateTime.Now;
82 blockTime = blockEnd - blockStart;
83 // Print the result to the user
84 Console.WriteLine($"Using iteration for depth and iteration for solution is {result}, it took {blockTime.TotalMilliseconds}");
85
86 Console.ForegroundColor = ConsoleColor.Cyan;
87 Console.WriteLine("Starting to process the solution using iteration for depth and counting for solution");
88 blockStart = DateTime.Now;
89 result = Logic.SolveCountingGCD(numbers);
90 blockEnd = DateTime.Now;
91 blockTime = blockEnd - blockStart;
92 // Print the result to the user
93 Console.WriteLine($"Using iteration for depth and counting for solution is {result}, it took {blockTime.TotalMilliseconds}");
94
95 Console.ForegroundColor = ConsoleColor.Blue;
96 Console.WriteLine("Starting to process the solution using recursion for depth and recursion for solution");
97 blockStart = DateTime.Now;
98 result = Logic.ArrayRecursionSolution(numbers, 0);
99 blockEnd = DateTime.Now;
100 blockTime = blockEnd - blockStart;
101 // Print the result to the user
102 Console.WriteLine($"Using recursion for depth and recursion for solution is {result}, it took {blockTime.TotalMilliseconds}");
103
104 Console.ForegroundColor = ConsoleColor.Green;
105 Console.WriteLine("Starting to process the solution using recursion for depth and iteration for solution");
106 blockStart = DateTime.Now;
107 result = Logic.ArrayIterationSolution(numbers, 0);
108 blockEnd = DateTime.Now;
109 blockTime = blockEnd - blockStart;
110 // Print the result to the user
111 Console.WriteLine($"Using recursion for depth and iteration for solution is {result}, it took {blockTime.TotalMilliseconds}");
112
113 Console.ForegroundColor = ConsoleColor.Magenta;
114 Console.WriteLine("Starting to process the solution using recursion for depth and counting for solution");
115 blockStart = DateTime.Now;
116 result = Logic.ArrayCountingSolution(numbers, 0);
117 blockEnd = DateTime.Now;
118 blockTime = blockEnd - blockStart;
119 // Print the result to the user
120 Console.WriteLine($"Using recursion for depth and counting for solution is {result}, it took {blockTime.TotalMilliseconds}");
121
122 Console.ResetColor();
123
124 Console.WriteLine("Preliminary method loading completed start tracking times of execution.");
125
126 //-----
127 // End Preliminary Untimed Execution Block
128
```

Figure 3-11: Preliminary timing block. This block runs all the methods but does not time them as in my tests I found that the first run method had extra overhead and accrued extra time for some reason so I removed them from the results for better consistency.

```
61
62 < Region Timing Blocks
63 < Preliminary Untimed Execution Block
131 < Region Tied Execution Block #1
132 //-----
133 // Start Tied Execution Block #1
134 //-----
135
136 RecursionCounting();
137
138 IterationCounting();
139
140 IterationRecursion();
141
142 IterationIteration();
143
144 RecursionRecursion();
145
146 RecursionIteration();
147
148 //-----
149 // End Tied Execution Block #1
150 //-----
151 < Region Tied Execution Block #2
152 //-----
153 // Start Tied Execution Block #2
154 //-----
155
156 IterationIteration();
157
158 RecursionCounting();
159
160 IterationCounting();
161
162 RecursionRecursion();
163
164 RecursionIteration();
165
166 IterationRecursion();
167
168 //-----
169 // End Tied Execution Block #2
170 //-----
171 < Region Tied Execution Block #3
172 //-----
173 // Start Tied Execution Block #3
174 //-----
175
176 IterationRecursion();
177
178
```

Figure 3-12: Screen shot of the first few timing blocks expanded. They are in different orders to avoid any weird call order funniness. I tried to keep the results as pure as I could.


```
DivisorLogic.cs - X
GreatestCommonDivisorRecursion - % GreatestCommonDivisorRecursion.Services.BusinessLogicLayer.DivisorLogic - % singlePairRecursiveSolution(int num1, int num2)
1 // Chad Galloway
2 // CST - 250 Programming in C# II
3 // 11/09/2020
4 // Greatest Common Divisor Recursion
5 // Activity 3 Part 3
6 // References:
7 //
8
9
10 namespace GreatestCommonDivisorRecursion.Services.BusinessLogicLayer
11 {
12     2 references
13     internal class DivisorLogic
14     {
15         /// <summary>
16         /// Solve a single pair of numbers in a complex GCD problem using recursion
17         /// </summary>
18         /// <param name="num1"></param>
19         /// <param name="num2"></param>
20         /// <returns></returns>
21         internal int singlePairRecursiveSolution(int num1, int num2)
22         {
23             // Declare and Initialize
24             int remainder = 0;
25
26             // Base Case: num2 = 0
27             if (num1 == 0 || num2 == 0)
28             {
29                 // Return the greatest common divisor
30                 return Math.Max(num1, num2);
31             }
32             else
33             {
34                 // Get the remainder of dividing num1 and num2
35                 remainder = num1 % num2;
36                 // Print an update to the user
37                 Console.WriteLine($"Not yet. The remainder of {num1} and {num2} is {remainder}.");
38                 // Call the recursive function of the second number and the remainder
39                 return singlePairRecursiveSolution(num2, remainder);
40             }
41         }
42
43         /// <summary> Solve a single pair of numbers in a complex GCD problem using iter...
44         2 references
45         internal int singlePairIterativeSolution(int num1, int num2)
46         {
47             // Declare and Initialize
48             int numerator = num1; // Probably not needed but it helped my brain see the math clearly in the loop below
49             int denominator = num2; // Probably not needed but it helped my brain see the math clearly in the loop below
50
51             // Catch Zero
52             if (num1 == 0 || num2 == 0)
53             {
54                 // Return the greatest common divisor
55                 return Math.Max(num1, num2);
56             }
57             else
58             {
59                 do
60                 {
61                     int temp = denominator; // store the current denominator temporarily
62                     denominator = numerator % denominator; // calculate a new denominator for the next iteration
63                     numerator = temp; // move the previous (temp) denominator up to the numerator
64                     Console.WriteLine($"Not yet. The remainder of {num1} and {num2} is {denominator}.");
65                 } while (denominator != 0);
66
67                 // The remainder of mod (%) was 0 so we have hit the greatest common divisor(denominator)
68                 // Return the numerator which is merely the temp storage of the previous (greatest common) divisor(denominator)
69                 return numerator;
70             }
71         }
72     }
73 }
```

Figure 3-15: Screen shot of the single pair GCD solution algorithms for recursion and iteration.

```
DivisorLogic.cs - X
GreatestCommonDivisorRecursion - % GreatestCommonDivisorRecursion.Services.BusinessLogicLayer.DivisorLogic - % singlePairRecursiveSolution(int num1, int num2)
78 /// <summary> solve a single pair of numbers in a complex GCD problem using coun...
79 2 references
80 internal int singlePairCountingSolution(int num1, int num2)
81 {
82     // Declare and Initialize
83     List<int> num1List = new List<int>();
84     List<int> num2List = new List<int>();
85
86     // Catch Zero
87     if (num1 == 0 || num2 == 0)
88     {
89         // Return the greatest common divisor
90         return Math.Max(num1, num2);
91     }
92
93     // Create list one
94     for (int i = 1; i <= num1; i++)
95     {
96         if (num1 % i == 0)
97         {
98             num1List.Add(i);
99         }
100     }
101     Console.WriteLine($"The first list has {num1List.Count} entries.");
102
103     // Create list two
104     for (int i = 1; i <= num2; i++)
105     {
106         if (num2 % i == 0)
107         {
108             num2List.Add(i);
109         }
110     }
111     Console.WriteLine($"The second list has {num2List.Count} entries.");
112
113     // Compare the lists with LINQ thanks ChatGPT
114     return num1List.Intersect<int>(num2List).DefaultIfEmpty().Max();
115 }
116
117
118
119 /// <summary>
120 /// Using iteration to probe the depth of the array of numbers while using recursion to solve the GCD problem
121 /// </summary>
122 /// <param name="numbers"></param>
123 /// <returns></returns>
124 2 references
125 internal int SolveRecursiveGCD(int[] numbers)
126 {
127     // Declare and Initialize
128     int divisor = Math.Abs(numbers[0]);
129
130     // Iterate through all numbers to get the GCD except for the first number, it was used as the first divisor.
131     for (int i = 1; i < numbers.Length; i++)
132     {
133         divisor = singlePairRecursiveSolution(Math.Abs(numbers[i]), divisor);
134     }
135
136     // Return the final result
137     return divisor;
138 }
```

Figure 3-16 Screen shot of the final pair solution using counting from one, as per the worksheet, and the first iteration depth method for recursion pairs.

```
DivisorLogic.cs X
GreatestCommonDivisorRecursion - %GreatestCommonDivisorRecursion.Services.BusinessLogic.DivisorLogic - %SinglePairRecursiveSolution(int num1, int num2)

136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178

/// <summary>
/// Using iteration to probe the depth of the array of numbers while using iteration to solve the GCD problem
/// </summary>
/// <param name="numbers"></param>
/// <returns></returns>
/// </summary>
internal int SolveIterativeGCD(int[] numbers)
{
    // Declare and Initialize
    int divisor = Math.Abs(numbers[0]);

    // Iterate through all numbers to get the GCD except for the first number, it was used as the first divisor.
    for (int i = 1; i < numbers.Length; i++)
    {
        divisor = singlePairIterativeSolution(Math.Abs(numbers[i]), divisor);
    }

    // Return the final result
    return divisor;
}

/// <summary>
/// Using iteration to probe depth and counting to solve the pairs
/// </summary>
/// <param name="numbers"></param>
/// <returns></returns>
/// </summary>
internal int SolveCountingGCD(int[] numbers)
{
    // Declare and Initialize
    int divisor = Math.Abs(numbers[0]);

    // Iterate through all numbers to get the GCD except for the first number, it was used as the first divisor.
    for (int i = 1; i < numbers.Length; i++)
    {
        divisor = singlePairCountingSolution(Math.Abs(numbers[i]), divisor);
    }

    // Return the final result
    return divisor;
}
```

Figure 3-17: Screen shot of the final two iteration depth methods for iteration and counting pairs methods.

```
DivisorLogic.cs X
GreatestCommonDivisorRecursion - %GreatestCommonDivisorRecursion.Services.BusinessLogic.DivisorLogic - %SolveRecursiveGCD(int[] numbers)

179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240

/// <summary>
/// Using recursion to probe the depth of each array of numbers while using recursion to solve the GCD for each pair
/// </summary>
/// <param name="numbers"></param>
/// <param name="index"></param>
/// <returns></returns>
/// </summary>
internal int arrayRecursionSolution(int[] numbers, int index)
{
    // Base case: The last element was reached, stop digging deeper into the array for more elements
    if (index == numbers.Length - 1)
    {
        Console.WriteLine($"{numbers[index]} is the last number in the array, collapsing the solution back down.");
        return numbers[index]; // Return the last array element (number) so the recursive calls can start collapsing back down to the solution.
    }

    // Returns a call to the next number pair in the array
    Console.WriteLine($"{Still more numbers. Calling the next number pair {numbers[index]} and {numbers[index + 1]}.");
    return singlePairRecursiveSolution(Math.Abs(numbers[index]), arrayRecursionSolution(numbers, index + 1));
}

/// <summary>
/// Use recursion to probe the depth of the array of numbers while using iteration to solve the GCD of each pair
/// </summary>
/// <param name="numbers"></param>
/// <param name="index"></param>
/// <returns></returns>
/// </summary>
internal int arrayIterationSolution(int[] numbers, int index)
{
    // Base case: The last element was reached, stop digging deeper into the array for more elements
    if (index == numbers.Length - 1)
    {
        Console.WriteLine($"{numbers[index]} is the last number in the array, collapsing the solution back down.");
        return numbers[index]; // Return the last array element (number) so the recursive calls can start collapsing back down to the solution.
    }

    // Returns a call to the next number pair in the array
    Console.WriteLine($"{Still more numbers. Calling the next number pair {numbers[index]} and {numbers[index + 1]}.");
    return singlePairIterativeSolution(Math.Abs(numbers[index]), arrayIterationSolution(numbers, index + 1));
}

/// <summary>
/// Use recursion to probe the depth and counting to solve the GCD of each pair
/// </summary>
/// <param name="numbers"></param>
/// <param name="index"></param>
/// <returns></returns>
/// </summary>
internal int arrayCountingSolution(int[] numbers, int index)
{
    // Base case: The last element was reached, stop digging deeper into the array for more elements
    if (index == numbers.Length - 1)
    {
        Console.WriteLine($"{numbers[index]} is the last number in the array, collapsing the solution back down.");
        return numbers[index]; // Return the last array element (number) so the recursive calls can start collapsing back down to the solution.
    }

    // Returns a call to the next number pair in the array
    Console.WriteLine($"{Still more numbers. Calling the next number pair {numbers[index]} and {numbers[index + 1]}.");
    return singlePairCountingSolution(Math.Abs(numbers[index]), arrayCountingSolution(numbers, index + 1));
}
```

Figure 3-18: Screen shot of the three recursive depth methods for iteration, recursion, and counting pairs.

/*

* Chad Galloway

* CST - 250 Programming in C# II

* 11/09/2025

* Greatest Common Denominator

* Activity 3 Part 3

* References:

*/

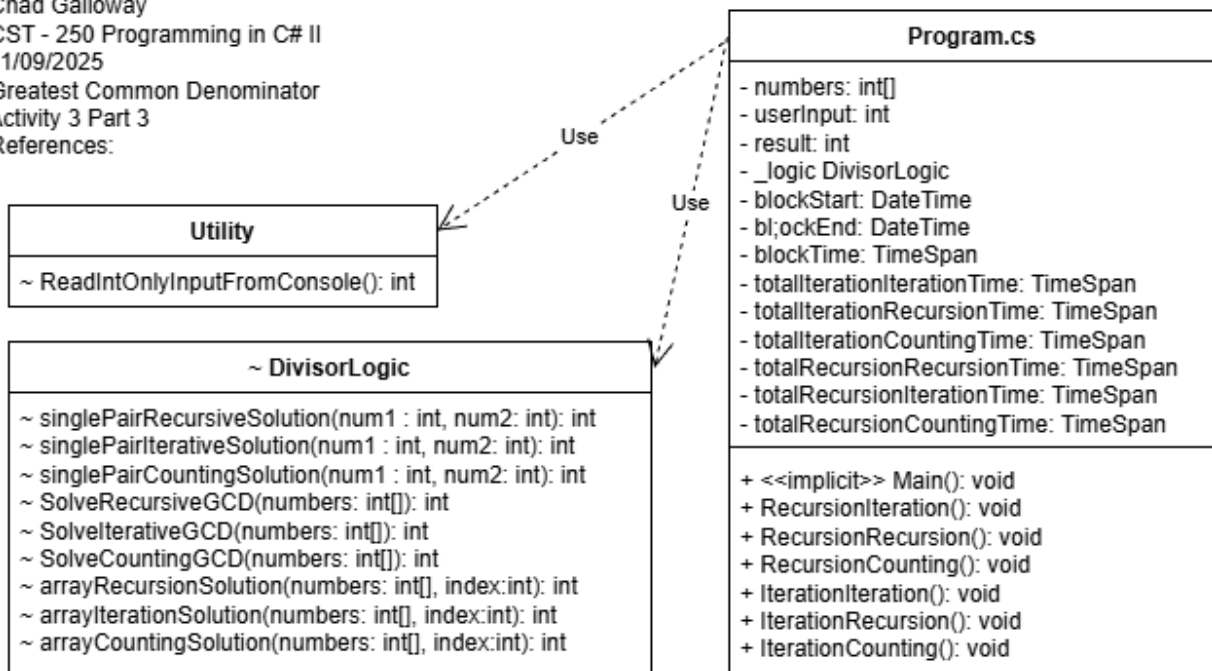


Figure 3-19: UML Changes for Part 3 GCD

Observations & Thinking

My train of thought on this set of challenges was based off of the tests I had done in the previous part when I was curious about the speeds. Adding the element of an array of numbers of unknown size made things interesting. I started by making the methods for finding the GCD of two numbers. I called these my single pairs. I created one of each type; recursion, iteration, and the counting up from one implementation described in the worksheet. With those sorted I had to figure out arrays so the user could enter more than two numbers. I knew I could easily loop through the array with a for loop and call whichever pair method I wanted to after that. What I was unsure of was whether I could use recursion for array traversal. As it turns out you can. By calling a nested recursion loop you can recursively traverse the array and at each step call the recursion method for figuring out pairs of numbers. After playing around with different methodologies I decided to run two branches, an iteration branch that would iterate through the array of numbers and call each pair function I had created, and a recursion branch that would call one of the pair methods with a call to itself inside one of the parameters for that particular pair method. Basically, the recursion traversal method would call the pair method which would need to call the traversal method which would call the pair method which would need to call the traversal method.... You get the point. This would continue until the last element in the array was reached and the traversal method would quit calling more pair methods and the stack would start collapsing solving the problem of GCD one pair at a time. After sorting out both array traversal methodologies I moved my focus to tracking time. My work in the previous part helped a great deal. I found, then, that the first method calls, for some reason (I guess the cost and time to allocate resources in Windows), take a considerable amount of time more and for this reason I discarded the times of each of the first calls for each method chain. I threw away the one call and then called each method type (there were six) 10 more times in mixed up orders to try and get the results as random and distributed as I could. I added in some color coding on the output to help tracking results and error better. What I found thru a lot of testing is that almost every time iteration depth with recursion pair solving was the fastest. Some times iteration depth and iteration pair solving would squeak out a small win. And on super rare occasions, usually on only two numbers iteration with counting pairs was fastest. What I did observe was that recursion array traversal was never the fastest method. Recursion traversal was consistently almost twice as slow as the iteration counter part on traversal. Traversal seems to matter a great deal on the time it takes but different pair solutions are a bit more forgiving on choice of solution method when time is the measuring stick.

PART 4

Flow Chart

/*
* Chad Galloway
* CST - 250 Programming in C# II
* 11/09/2025
* Flood Fill
* Activity 3 Part 4
* References:
*/

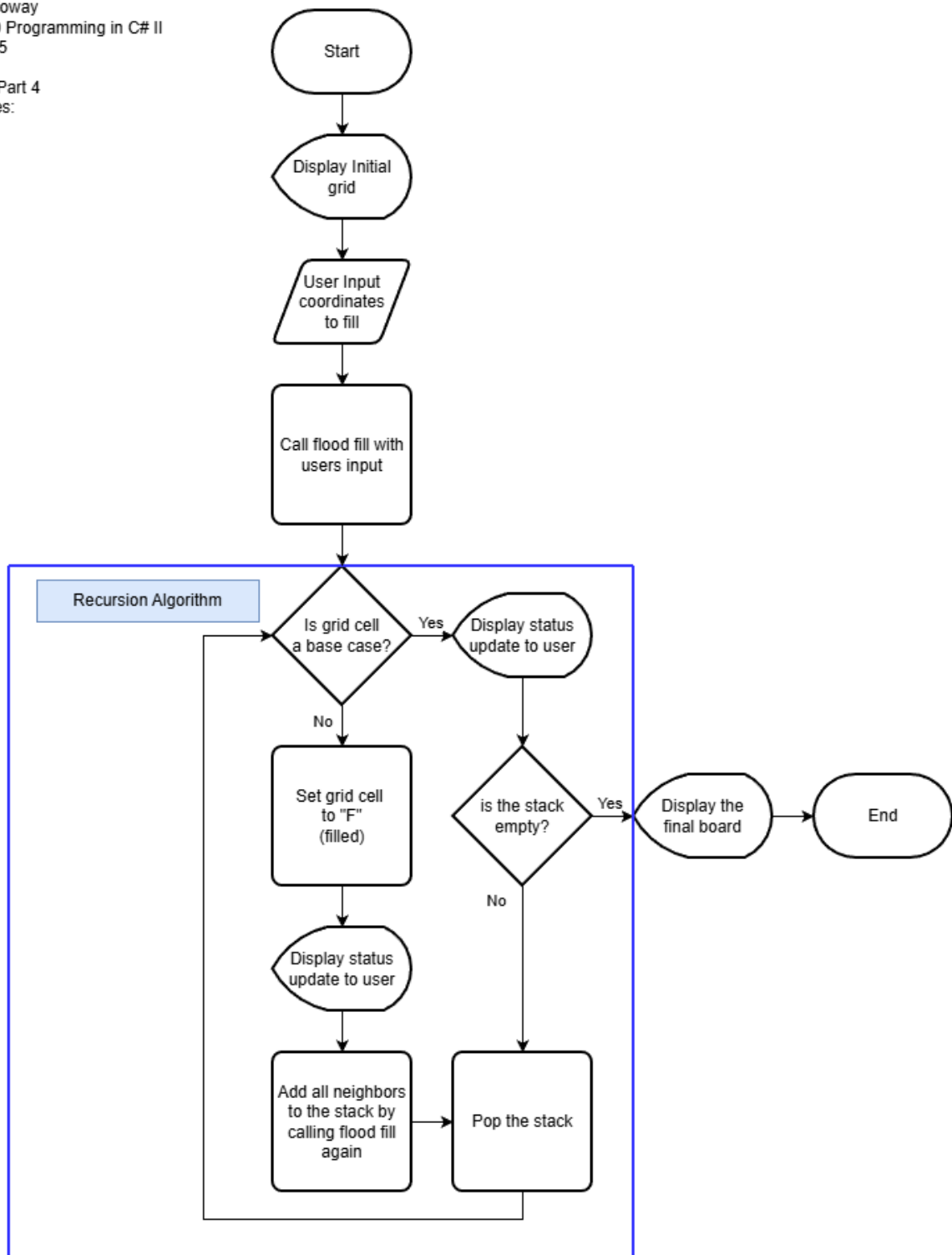


Figure 4-1: Flow chart of Activity 3 Part 4 flood fill recursion

UML Class Diagram

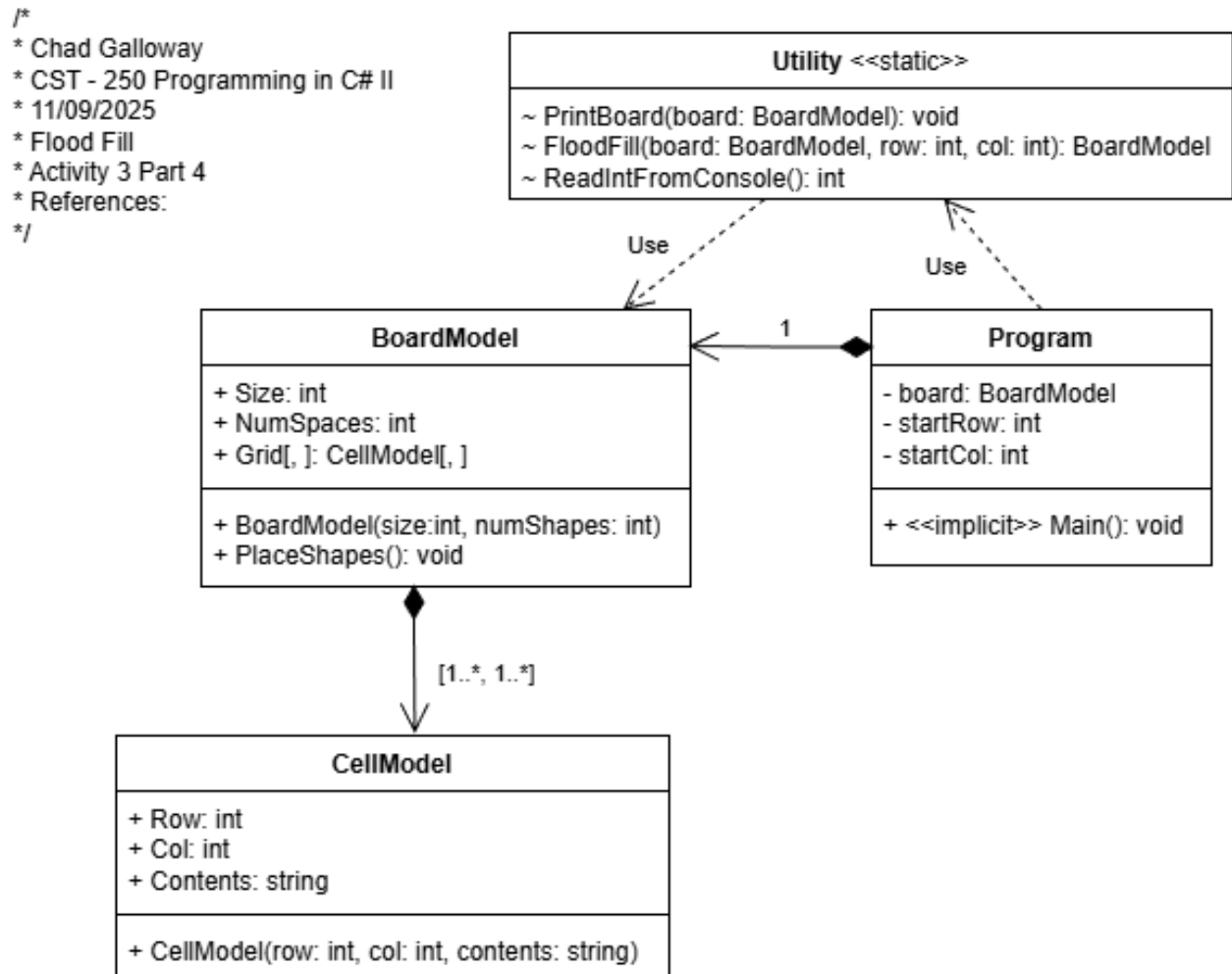
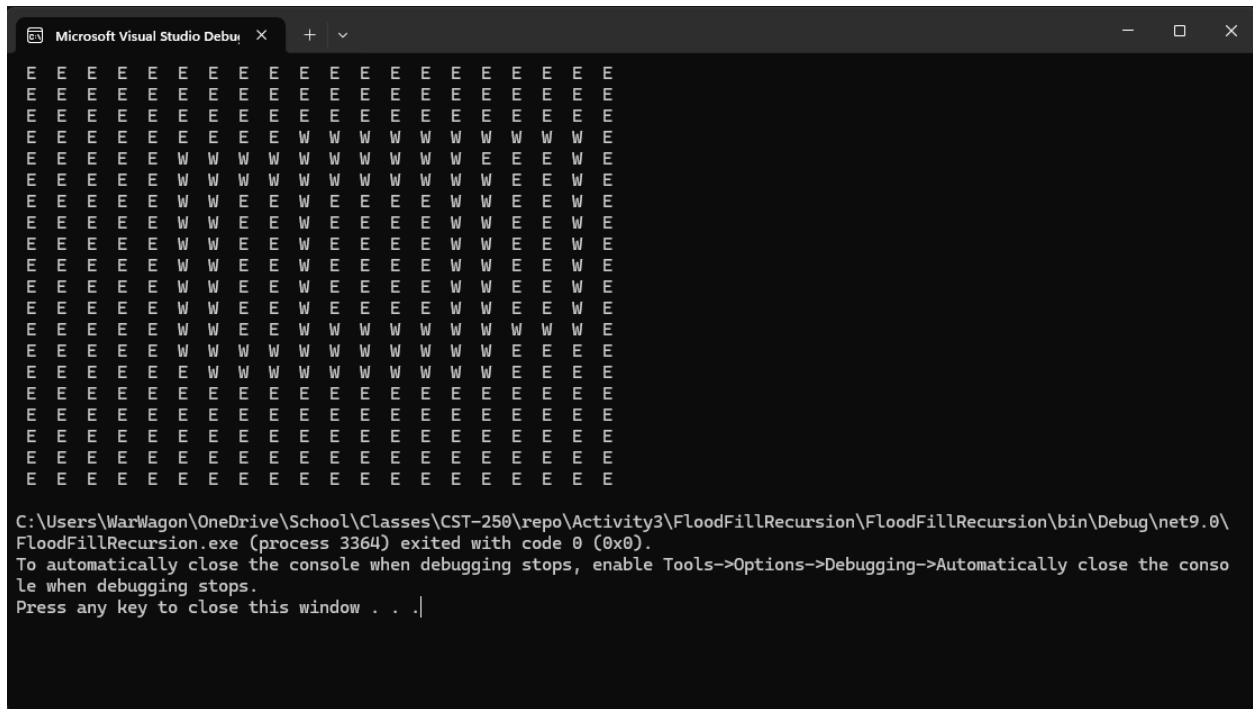


Figure X: UML Class Diagram for Activity 3 part 4

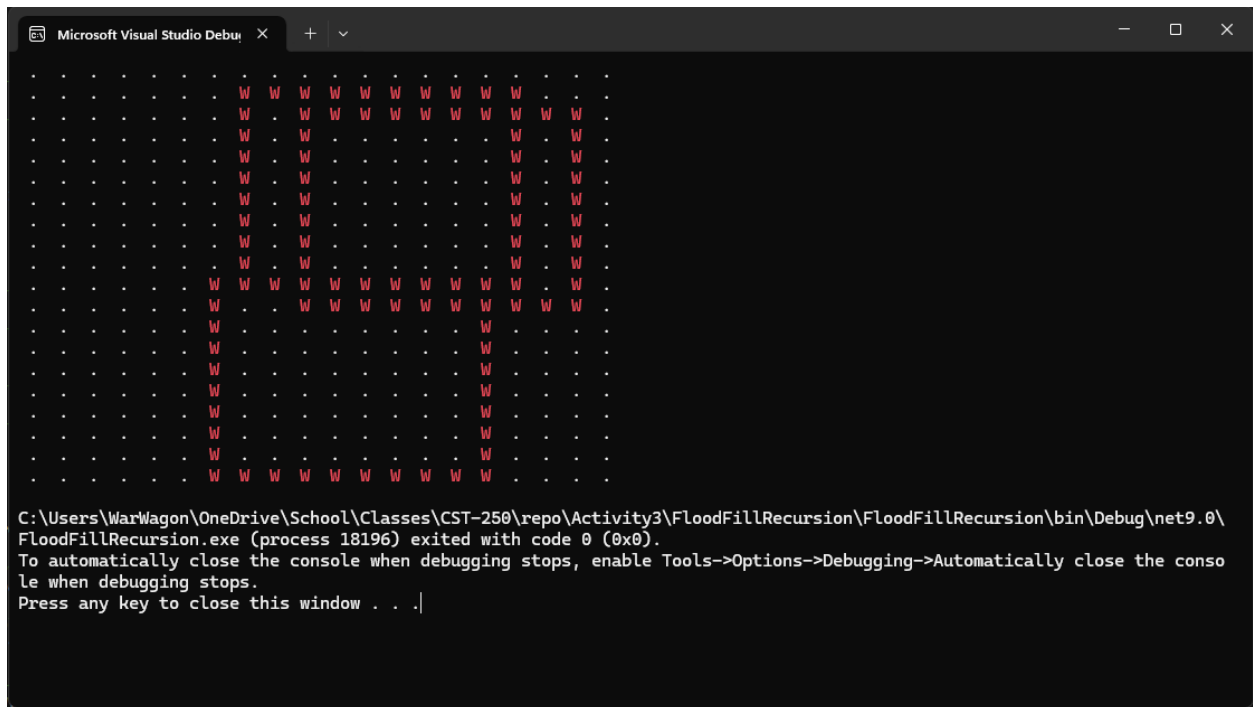
Screen Shots



The screenshot shows the Microsoft Visual Studio Debug Console window. The main area displays a 20x20 grid of characters. The first 18 rows consist of 18 'E' characters followed by 2 'W' characters. The last two rows consist of 18 'W' characters followed by 2 'E' characters. Below the grid, the following text is displayed:

```
C:\Users\WarWagon\OneDrive\School\Classes\CST-250\repo\Activity3\FloodFillRecursion\FloodFillRecursion\bin\Debug\net9.0\FloodFillRecursion.exe (process 3364) exited with code 0 (0x0).  
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops.  
Press any key to close this window . . .|
```

Figure 4-3: Part 4 App Running for the first time



The screenshot shows the Microsoft Visual Studio Debug Console window. The main area displays a 20x20 grid of characters. The first 18 rows consist of 18 'W' characters followed by 2 'E' characters. The last two rows consist of 18 'E' characters followed by 2 'W' characters. Below the grid, the following text is displayed:

```
C:\Users\WarWagon\OneDrive\School\Classes\CST-250\repo\Activity3\FloodFillRecursion\FloodFillRecursion\bin\Debug\net9.0\FloodFillRecursion.exe (process 18196) exited with code 0 (0x0).  
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops.  
Press any key to close this window . . .|
```

Figure 4-4: Part 4 App Running for the second time with colored shapes.

```
Microsoft Visual Studio Debug Console
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20
1 W W W W W W W W W W W W W W W W W W W W W
2 W . W W W W W W W W W W W W W W W W W W W
3 W . W . . . . . W W W W W W W W W W W W W W
4 W . W . . . . . W W W W W W W W W W W W W W
5 W . W . . . . . W W W W W W W W W W W W W W
6 W . W . . . . . W W W W W W W W W W W W W W
7 W . W . . . . . W W W W W W W W W W W W W W
8 W . W . . . . . W W W W W W W W W W W W W W
9 W . W . . . . . W W W W W W W W W W W W W W
10 W W W W W W W W W W W W W W W W W W W W W
11 . . W W W W W W W W W W W W W W W W W W W
12 . . . . . . W . . . . . . . . . W W W W W W
13 . . . . . . W . . . . . . . . . W W W W W W
14 . . . . . . W W W W W W W W W W W W W W W W
15 . . . . . . . . . . . . . . . . . . . . . .
16 . . . . . . . . . . . . . . . . . . . . . .
17 . . . . . . . . . . . . . . . . . . . . . .
18 . . . . . . . . . . . . . . . . . . . . . .
19 . . . . . . . . . . . . . . . . . . . . . .
20 . . . . . . . . . . . . . . . . . . . . . .

C:\Users\WarWagon\OneDrive\School\Classes\CST-250\repo\Activity3\FloodFillRecursion\FloodFillRecursion\bin\Debug\net9.0\FloodFillRecursion.exe (process 16432) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops.
Press any key to close this window . . .|
```

Figure 4-5: The addition of row and column number.

```
Microsoft Visual Studio Debug Console
9 . . W W W W W W W W W W W W W W W W W W W W
10 . . W . W . . W . . . W . W . . W . . . .
11 . . W . W . . W . . . W . W . . W . . . .
12 . . W . W . . W . . . W . W . . W . . . .
13 . . W . W W W W W W W W W W W W W W W W W W
14 . . W . . . . W . . . W . . . . W W W W W W
15 . . W . . . . W . . . W . . . . W W W W W W
16 . . W . . . . W W W W W W W W W W W W W W W
17 . . W . . . . W . . . W . . . . W W W W W W
18 . . W W W W W W W W W W W W W W W W W W W W
19 . . . . . . . . . . . . . . . . . . . . . .
20 . . . . . . . . . . . . . . . . . . . . . .
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20
1 ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~
2 ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~
3 ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~
4 ~ ~ ~ ~ W W W W W W W W W W W W W W W W W
5 ~ ~ ~ ~ . . . . . . . . . . . . . . . . .
6 ~ ~ ~ ~ W . . . . . . . . . . W ~ ~ ~ ~ ~
7 ~ ~ ~ ~ W . . W W W W W W W W W W W W W W
8 ~ ~ ~ ~ W . . W . . . . . . . . . . ~ ~ ~
9 ~ ~ ~ ~ W W W W W W W W W W W W W W W W W
10 ~ ~ ~ ~ W . W . . W . . . W . W . . W ~ ~
11 ~ ~ ~ ~ W . W . . W . . . W . W . . W ~ ~
12 ~ ~ ~ ~ W . W . . W . . . W . W . . W ~ ~
13 ~ ~ ~ ~ W . W W W W W W W W W W W W W W W
14 ~ ~ ~ ~ . . . . W . . . W . . . . W ~ ~ ~
15 ~ ~ ~ ~ W . . . . W . . . W . . . . W ~ ~
16 ~ ~ ~ ~ W . . . . W W W W W W W W W W W W
17 ~ ~ ~ ~ W . . . . W . . . W . . . . W ~ ~
18 ~ ~ ~ ~ W W W W W W W W W W W W W W W W W
19 ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~
20 ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~
```

Figure 4-6: Flood fill implemented with blue tilde.

```
Microsoft Visual Studio Debug Console
East to: location 8, 9: Already Filled. Stop
South to: location 9, 8: Already Filled. Stop
West to: location 8, 7: Already Filled. Stop
 1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20
1 . . . . . . . . . . . . . . . . . . . .
2 . . . . . . . . . . . . . . . . . . . .
3 . . W W W W W W W W W W . . . . . . . .
4 . . W . . . . . . . . . . W . . . . . .
5 . . W . . W W W W W W W W W W W W W W W W
6 . . W . . W ~ ~ ~ ~ W W . . W . . . . W
7 . . W . . W ~ ~ ~ ~ W W . . W . . . . W
8 . . W . . W ~ ~ ~ ~ W W . . W . . . . W
9 . . W . . W ~ ~ ~ ~ W W . . W . . . . W
10 . . W . . W ~ ~ ~ ~ W W . . W . . . . W
11 . . W . . W ~ ~ ~ ~ W W . . W . . . . W
12 . . W W W W W W W W W W . . W . . . . W
13 . . . . W . . . . W . . W . . . . W
14 . . . . W W W W W W W W W W W W W W W W
15 . . . . . . . . . . . . . . . . . . . .
16 . . . . . . . . . . . . . . . . . . . .
17 . . . . . . . . . . . . . . . . . . . .
18 . . . . . . . . . . . . . . . . . . . .
19 . . . . . . . . . . . . . . . . . . . .
20 . . . . . . . . . . . . . . . . . . . .

C:\Users\WarWagon\OneDrive\School\Classes\CST-250\repo\Activity3\FloodFillRecursion\FloodFillRecursion\bin\Debug\net9.0\FloodFillRecursion.exe (process 7696) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops.
Press any key to close this window . . .|
```

Figure 4-7: Completed implementing user selected flood fill start location

Code

```
Program.cs - X
FloodFillRecursion - Utility - PrintBoard(BoardModel board)

1  //
2  // Chad Galloway
3  // C# - 250 Programming in C# II
4  // 12/09/2020
5  // Flood Fill Recursion
6  // Activity 3 Part 4
7  // References:
8  //
9
10 using FloodFillRecursion.Models;
11
12 //-----
13 // Start of the Main Method
14 //-----
15
16 // Declare and Initialize
17 // Create a new BoardModel
18 BoardModel board = new BoardModel(20, 3);
19 int startRow = -1, startCol = -1;
20
21 // Print the board to the console
22 Utility.PrintBoard(board);
23
24 // Prompt the user for the starting row (1 - 20)
25 Console.WriteLine("Enter the row to start the flood fill at: ");
26 // Remove 1 from the input to get 0-19 range for row
27 startRow = Utility.ReadIntFromConsole() - 1;
28
29 // Prompt the user for the starting column (1 - 20)
30 Console.WriteLine("Enter the column to start the flood fill at: ");
31 // Remove 1 from the input to get 0-19 range for col
32 startCol = Utility.ReadIntFromConsole() - 1;
33
34 // Call the flood fill method using the board
35 board = Utility.FloodFill(board, startRow, startCol);
36 // Print the board to the console
37 Utility.PrintBoard(board);
38
39 //-----
40 // End of the Main Method
41 //-----
42
```

Figure 4-8: Code Citations and the main method.

```
Program.cs - X
FloodFillRecursion - Utility - ReadIntFromConsole()

48 3 references
49 static class Utility
50 {
51     /// <summary>
52     /// Print the board to the console
53     /// </summary>
54     /// <param name="board">BoardModel</param>
55     /// </param>
56     2 references
57     internal static void PrintBoard(BoardModel board)
58     {
59         // Make sure the color of the column numbers is white
60         Console.ForegroundColor = ConsoleColor.White;
61         // Start the column numbers row with a space to keep the numbers aligned
62         Console.WriteLine(" ");
63         // Loop to add column numbers for the board
64         for (int column = 0; column < board.Size; column++)
65         {
66             // Print the column with a 2-character width
67             Console.Write($"{column + 1, 2}");
68             Console.WriteLine();
69         }
70         // Loop through the rows of the board
71         for (int row = 0; row < board.Size; row++)
72         {
73             // Print each row number in white
74             Console.ForegroundColor = ConsoleColor.White;
75             Console.WriteLine($"{row + 1, 2}");
76             // Loop through the columns of the board
77             for (int col = 0; col < board.Size; col++)
78             {
79                 // Check if the current cell is a wall
80                 if (board.Grid[row, col].Contents == "W")
81                 {
82                     // Change the text color to red
83                     Console.ForegroundColor = ConsoleColor.Red;
84                     Console.WriteLine(" W ");
85                 }
86                 // Check if the current cell is an end point
87                 else if (board.Grid[row, col].Contents == "E")
88                 {
89                     // Change the text color to white
90                     Console.ForegroundColor = ConsoleColor.White;
91                     Console.WriteLine(" E ");
92                 }
93                 else if (board.Grid[row, col].Contents == "F")
94                 {
95                     Console.ForegroundColor = ConsoleColor.DarkBlue;
96                     Console.WriteLine(" F ");
97                 }
98                 // Otherwise, it's an empty path
99                 else
100                 {
101                     Console.WriteLine(" ");
102                 }
103             }
104             // Use a write line to start a new row
105             Console.WriteLine();
106         }
107         Console.ResetColor();
108     }
109     /// <summary> Perform a flood fill algorithm on the given row and col
110     1 reference
111     internal static BoardModel FloodFill(BoardModel board, int row, int col) // End of FloodFill method
112     {
113         // <summary> Read an integer number from the console
114         2 references
115         internal static int ReadIntFromConsole()
116     {
117         }
118     }
119 }
120
```

Figure 4-9: Utility class and the print board method.

```

109 // Summary
110 // Perform a flood fill algorithm on the given row and col
111 // Summary
112 // param name "board"
113 // returns
114 // returns
115 internal static BoardModel FloodFill(BoardModel board, int row, int col)
116 {
117     // Declare and Initialize
118     int sleepCount = 100; // milliseconds
119
120     // Change text color to white
121     Console.ForegroundColor = ConsoleColor.White;
122     // Print the current cell to the console
123     Console.WriteLine($"Location {row}, {col}");
124     // Pause the program for sleepCount number of milliseconds
125     Thread.Sleep(sleepCount);
126
127     // Check if the cell is on the board
128     if (row < 0 || row >= board.Size || col < 0 || col >= board.Size)
129     {
130         // Print a message indicating the cell is out of bounds
131         Console.WriteLine("Out of bounds. Stop");
132         // Pause the program for sleepCount number of milliseconds
133         Thread.Sleep(sleepCount);
134
135         // If the cell is not on the board, end the method
136         return board;
137     }
138
139     // If the cell is a wall, end the method
140     if (board.Grid[row, col].Contents == "W")
141     {
142         // Print a message indicating the cell is a wall
143         Console.WriteLine("Hit a wall. Stop");
144         // Pause the program for sleepCount number of milliseconds
145         Thread.Sleep(sleepCount);
146
147         return board;
148     }
149
150     // If the cell has already been filled, end the method
151     else if (board.Grid[row, col].Contents == "F")
152     {
153         // Print a message indicating the cell already filled
154         Console.WriteLine("Already Filled. Stop");
155         // Pause the program for sleepCount number of milliseconds
156         Thread.Sleep(sleepCount);
157
158         return board;
159     }
160
161     // Else, fill the cell
162     else
163     {

```

Figure 4-10: Flood Fill Initializations and base cases.

```

164     board.Grid[row, col].Contents = "F";
165     // Print a message indicating the cell is filling up
166     Console.WriteLine("Filling");
167     // Pause the program for sleepCount number of milliseconds
168     Thread.Sleep(sleepCount + sleepCount);
169     Console.WriteLine("");
170     // Pause the program for sleepCount number of milliseconds
171     Thread.Sleep(sleepCount + sleepCount);
172     Console.WriteLine("");
173     // Pause the program for sleepCount number of milliseconds
174     Thread.Sleep(sleepCount + sleepCount);
175     Console.WriteLine("");
176     // Pause the program for sleepCount number of milliseconds
177     Thread.Sleep(sleepCount + sleepCount);
178
179     // Improve the visual effect of the flood fill
180     // Comment out to have program history
181     Console.Clear();
182
183     // Print the current board
184     Console.WriteLine();
185     PrintBoard(board);
186
187     // Print a message indicating the next flood fill direction
188     Console.ForegroundColor = ConsoleColor.Blue;
189     Console.WriteLine("North to: ");
190     // Call the flood fill method to the north
191     board = FloodFill(board, row - 1, col);
192
193     // Print a message indicating the next flood fill direction
194     Console.ForegroundColor = ConsoleColor.Yellow;
195     Console.WriteLine("East to: ");
196     // Call the flood fill method to the east
197     board = FloodFill(board, row, col + 1);
198
199     // Print a message indicating the next flood fill direction
200     Console.ForegroundColor = ConsoleColor.Magenta;
201     Console.WriteLine("South to: ");
202     // Call the flood fill method to the south
203     board = FloodFill(board, row + 1, col);
204
205     // Print a message indicating the next flood fill direction
206     Console.ForegroundColor = ConsoleColor.Cyan;
207     Console.WriteLine("West to: ");
208     // Call the flood fill method to the west
209     board = FloodFill(board, row, col - 1);
210
211     // Return the board
212     return board;
213 } // End of FloodFill method

```

Figure 4-11: Flood Fill recursion calls and visual effects

Figure 4-12: ReadIntFromConsole() method from the Utility class

Figure 4-13: Board model class citations and constructor

Figure 4-14: Board model place shapes method

```

CellModels: a X
FloodFillRecursion
  1
  2
  3
  4
  5
  6
  7
  8
  9
  10
  11
  12
  13
  14
  15
  16
  17
  18
  19
  20
  21
  22
  23
  24
  25
  26
  27
  28
  29
  30
  31
  32
  33
  Chad Galloway
  * C#7 - 2ND Programming in C# II
  * 11/09/2020
  * Flood Fill Recursion
  * Activity 3 Part 4
  * References:
  namespace FloodFillRecursion.Models
  {
    4 references
    internal class CellModel
    {
      // Cell Model Properties
      // reference
      public int Row { get; set; }
      // reference
      public int Col { get; set; }
      // reference
      public string Contents { get; set; }

      /// <summary>
      /// Parameterized constructor for CellModel
      /// </summary>
      /// <param name="row"></param>
      /// <param name="col"></param>
      /// <param name="contents"></param>
      // reference
      public CellModel(int row, int col, string contents)
      {
        Row = row;
        Col = col;
        Contents = contents;
      }
    }
  }

```

Figure 4-15: Cell model class citations and constructor

```

BoardModels: a X
FloodFillRecursion
  44
  45
  46
  47
  48
  49
  50
  51
  52
  53
  54
  55
  56
  57
  58
  59
  60
  61
  62
  63
  64
  65
  66
  67
  68
  69
  70
  71
  72
  73
  74
  75
  76
  /// <summary>
  /// Create shape to place on the board
  /// </summary>
  // reference
  public void PlaceShapes()
  {
    // Declare and Initialize
    // Random object to generate numbers
    Random random = new Random();
    int shapeSize = Size / 2, row = 0, col = 0;

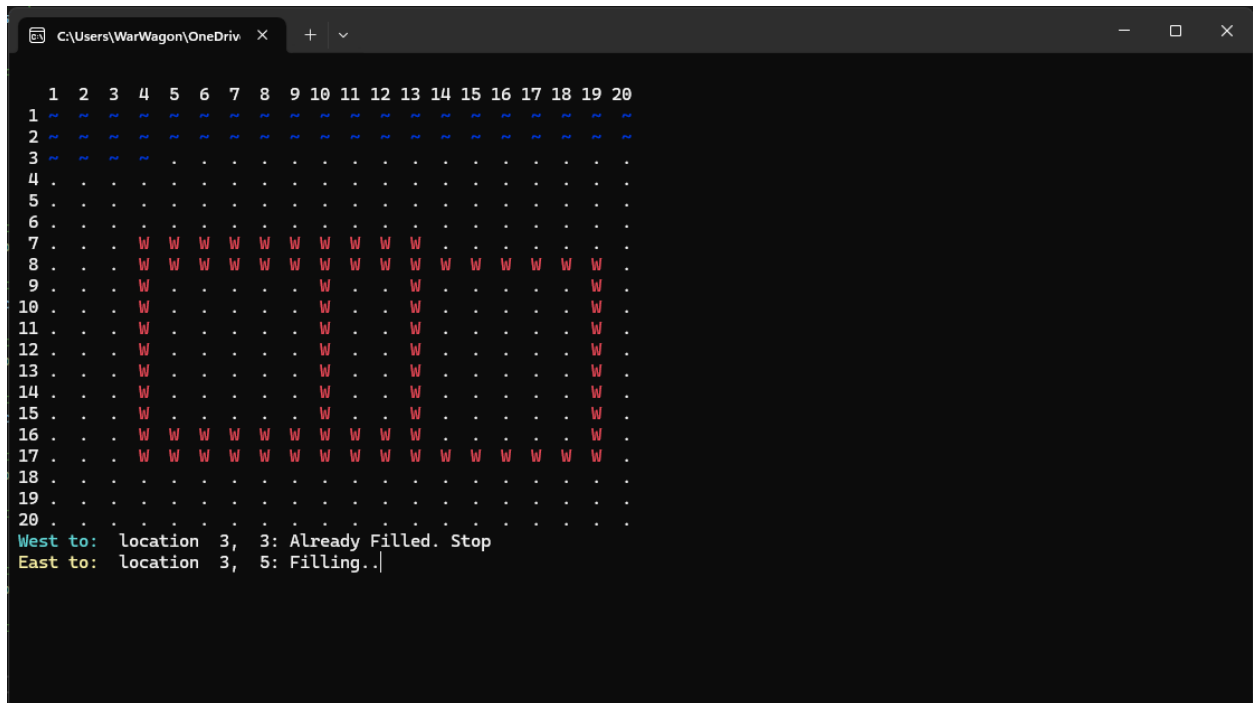
    // Create three shapes
    for (int shapes = 0; shapes < NumShapes; shapes++)
    {
      // Generate the row and col for the
      // top left corner of the square
      row = random.Next(0, Size - shapeSize + 1);
      col = random.Next(0, Size - shapeSize + 1);
      for (int offset = 0; offset < shapeSize; offset++)
      {
        // Top wall
        Grid[row, col + offset].Contents = "W";
        // Bottom wall
        Grid[row + shapeSize - 1, col + offset].Contents = "W";
        // Left wall
        Grid[row + offset, col].Contents = "W";
        // Right wall
        Grid[row + offset, col + shapeSize - 1].Contents = "W";
      }
    }
  } // End of PlaceShapes method

```

Figure 4-16: UML for Flood Fill Project

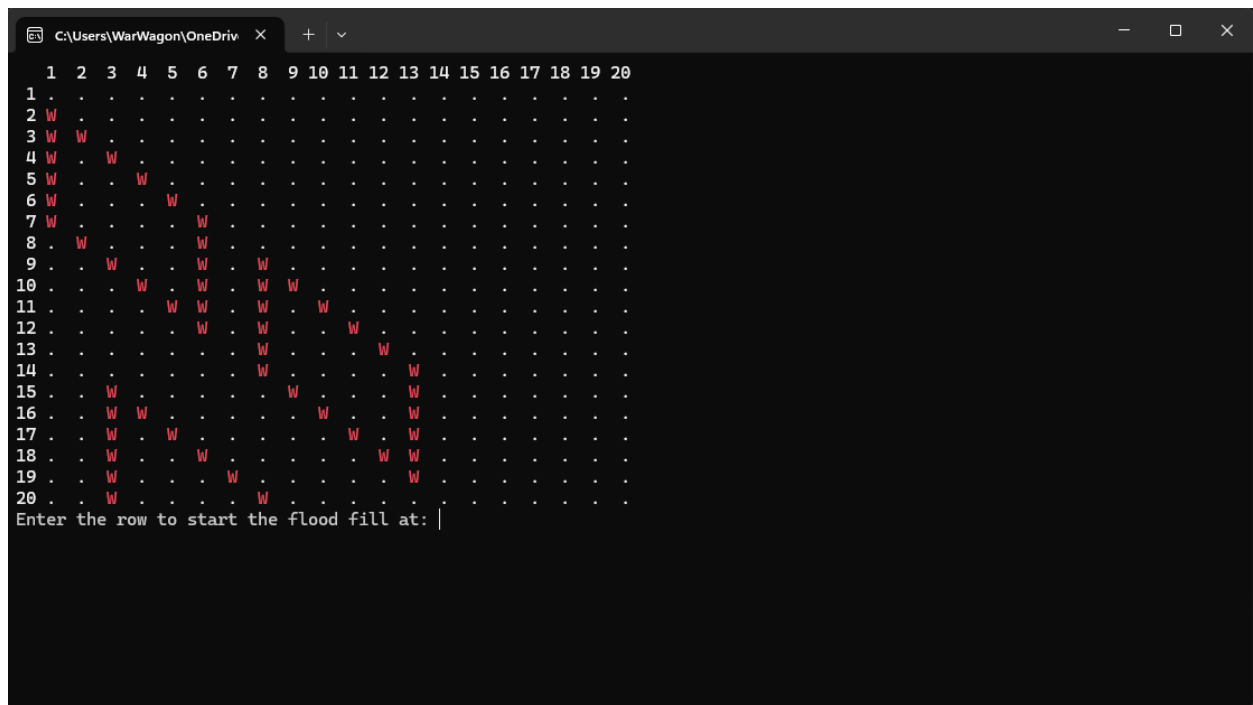
PART 4 CHALLENGES

Screen Shots



```
C:\Users\WarWagon\OneDrive >
 1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20
1  w w w w w w w w w w w w w w w w w w w w
2  w w w w w w w w w w w w w w w w w w w w
3  w w w w w w w w w w w w w w w w w w w w
4  . . . . . . . . . . . . . . . . . . . .
5  . . . . . . . . . . . . . . . . . . . .
6  . . . . . . . . . . . . . . . . . . . .
7  . . . w w w w w w w w w w w w w w w w w
8  . . . w w w w w w w w w w w w w w w w w
9  . . . w . . . . . w . . . w . . . w . . .
10 . . . w . . . . . w . . . w . . . w . . .
11 . . . w . . . . . w . . . w . . . w . . .
12 . . . w . . . . . w . . . w . . . w . . .
13 . . . w . . . . . w . . . w . . . w . . .
14 . . . w . . . . . w . . . w . . . w . . .
15 . . . w . . . . . w . . . w . . . w . . .
16 . . . w w w w w w w w w w w w w w w w w
17 . . . w w w w w w w w w w w w w w w w w
18 . . . . . . . . . . . . . . . . . . . .
19 . . . . . . . . . . . . . . . . . . . .
20 . . . . . . . . . . . . . . . . . . . .
West to: location 3, 3: Already Filled. Stop
East to: location 3, 5: Filling..|
```

Figure 4-17: Challenge one running west, east, south, then north.



```
C:\Users\WarWagon\OneDrive >
 1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20
1  . . . . . . . . . . . . . . . . . . . .
2  w . . . . . . . . . . . . . . . . . . . .
3  w w . . . . . . . . . . . . . . . . . . . .
4  w . w . . . . . . . . . . . . . . . . . . . .
5  w . . w . . . . . . . . . . . . . . . . . . . .
6  w . . . w . . . . . . . . . . . . . . . . . . . .
7  w . . . . w . . . . . . . . . . . . . . . . . . . .
8  . w . . . w . . . . . . . . . . . . . . . . . . . .
9  . . w . . w . w . . . . . . . . . . . . . . . . . . . .
10 . . . w . w . w w . . . . . . . . . . . . . . . . . . . .
11 . . . . w w . w w . . . . . . . . . . . . . . . . . . . .
12 . . . . . w . w . . . w . . . . . . . . . . . . . . . . . . . .
13 . . . . . . . w . . . w . . . . . . . . . . . . . . . . . . . .
14 . . . . . . . w . . . w . . . . . . . . . . . . . . . . . . . .
15 . . . w . . . . . w . . . . . w . . . . . . . . . . . . . . . . . . . .
16 . . . w w . . . . . w . . . . . w . . . . . . . . . . . . . . . . . . . .
17 . . . w . w . . . . . w . . . . . w . . . . . . . . . . . . . . . . . . . .
18 . . . w . . w . . . . . w . . . . . w w . . . . . . . . . . . . . . . . . . . .
19 . . . w . . . . w . . . . . w . . . . . w . . . . . . . . . . . . . . . . . . . .
20 . . . w . . . . . w . . . . . . . . . . . . . . . . . . . . . . . . . .
Enter the row to start the flood fill at: |
```

Figure 4-18: Challenge two spawning different shapes

```
Microsoft Visual Studio Debug Console
North to: location 10, 9: Hit a wall. Stop
East to: location 12, 11: Hit a wall. Stop
South to: location 13, 10: Already Filled. Stop
North to: location 11, 10: Hit a wall. Stop
South to: location 14, 9: Already Filled. Stop
North to: location 12, 9: Already Filled. Stop
East to: location 14, 11: Already Filled. Stop
South to: location 15, 10: Already Filled. Stop
North to: location 13, 10: Already Filled. Stop
 1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20
1  .  .  .  .  .  .  .  .  .  .  .  .  .  .  .  .  .  .  .  .
2  W  .  .  .  .  .  .  .  .  .  .  .  .  .  .  .  .  .  .  .
3  W  W  .  .  .  .  .  .  .  .  .  .  .  .  .  .  .  .  .
4  W  .  W  .  .  .  .  .  .  .  .  .  .  .  .  .  .  .  .
5  W  .  .  W  .  .  .  .  .  .  .  .  .  .  .  .  .  .  .
6  W  .  .  .  W  .  .  .  .  .  .  .  .  .  .  .  .  .  .
7  W  .  .  .  .  W  .  .  .  .  .  .  .  .  .  .  .  .  .
8  .  W  .  .  .  .  W  .  .  .  .  .  .  .  .  .  .  .  .
9  .  .  W  .  .  .  .  W  .  .  .  .  .  .  .  .  .  .  .
10 .  .  .  W  .  .  .  .  W  W  .  .  .  .  .  .  .  .  .
11 .  .  .  .  W  W  .  W  .  W  .  .  .  .  .  .  .  .  .
12 .  .  .  .  .  W  .  W  .  .  W  .  .  .  .  .  .  .  .
13 .  .  .  .  .  .  W  .  .  .  .  W  .  .  .  .  .  .  .
14 .  .  .  .  .  .  .  W  .  .  .  .  .  W  .  .  .  .  .
15 .  .  W  .  .  .  .  .  W  .  .  .  .  .  .  W  .  .  .
16 .  W  W  .  .  .  .  .  .  W  .  .  .  .  .  .  .  W  .
17 .  .  W  .  W  .  .  .  .  .  W  .  .  .  .  .  .  .  W
18 .  W  .  .  W  .  .  .  .  .  .  .  W  .  .  .  .  .  .
19 .  W  .  .  .  W  .  .  .  .  .  .  .  W  .  .  .  .  .
20 .  .  W  .  .  .  .  W  .  .  .  .  .  .  .  .  .  .  .
```

Figure 4-19: Challenge two fill completed. No jumping over diagonals with original north south east west; four direction flooding.

```
C:\Users\WarWagon\OneDrive
 1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20
1  .  .  .  .  .  .  .  .  .  .  .  .  .  .  .  .  .  .  .  .
2  .  .  .  .  .  .  .  .  .  .  .  .  .  .  .  .  .  .  .  .
3  W  .  .  .  .  .  .  .  .  .  .  .  .  .  .  .  .  .  .  .
4  W  W  .  .  .  .  .  .  .  .  .  .  .  .  .  .  .  .  .
5  W  .  W  .  .  .  .  .  .  .  .  .  .  .  .  .  .  .  .
6  W  .  .  W  .  .  .  .  .  .  .  .  .  .  .  .  .  .  .
7  W  .  .  .  W  .  .  .  .  .  .  .  W  .  .  .  .  .  .
8  W  .  .  .  .  W  .  .  .  .  .  .  .  W  W  .  .  .  .
9  .  W  .  .  .  W  .  .  .  .  .  .  .  W  .  W  .  .  .
10 .  .  W  .  .  W  .  W  .  .  .  .  W  .  .  W  .  .  .
11 .  .  .  W  .  W  .  W  W  .  .  .  W  .  .  .  W  .  .
12 .  .  .  .  W  W  .  W  .  W  .  .  W  .  .  .  .  W  .
13 .  .  .  .  .  W  .  W  .  .  W  .  W  .  .  .  W  .  .
14 .  .  .  .  .  .  W  .  .  .  W  .  .  W  .  .  .  W  .
15 .  .  .  .  .  .  .  W  .  .  .  .  W  .  W  .  W  .  .
16 .  .  .  .  .  .  .  .  W  .  .  .  W  .  .  W  W  .  .
17 .  .  .  .  .  .  .  .  .  W  W  .  W  .  .  .  W  .  .
18 .  .  .  .  .  .  .  .  .  .  W  W  .  W  .  .  .  .  W
19 .  .  .  .  .  .  .  .  .  .  .  W  W  .  W  .  .  .  .
20 .  .  .  .  .  .  .  .  .  .  .  .  W  W  .  W  .  .  .  .
West to: location 17, 10: Hit a wall. Stop
East to: location 17, 12: Already Filled. Stop
South to: location 18, 11: Hit a wall. Stop
North to: location 16, 11: Already Filled. Stop
NorthWest to: location 16, 10: Already Filled. Stop
NorthEast to: location 16, 12: Already Filled. Stop
SouthWest to: location 18, 10: Already Filled. Stop
SouthEast to: location 18, 12: Already Filled. Stop
```

Figure 4-20: Challenge three fill running. Jumping over diagonals happens with calls to NorthWest and the other corners giving eight direction flooding.

Code

```

167         Thread.Sleep(sleepCount + sleepCount);
168         Console.WriteLine("");
169         // Pause the program for sleepCount number of milliseconds
170         Thread.Sleep(sleepCount + sleepCount);
171         Console.WriteLine("");
172         // Pause the program for sleepCount number of milliseconds
173         Thread.Sleep(sleepCount + sleepCount);
174         Console.WriteLine("");
175         // Pause the program for sleepCount number of milliseconds
176         Thread.Sleep(sleepCount + sleepCount);
177     }
178
179     // Improve the visual effect of the flood fill
180     // Comment out to have program history
181     Console.Clear();
182
183     // Print the current board
184     Console.WriteLine();
185     PrintBoard(board);
186
187     // Print a message indicating the next flood fill direction
188     Console.ForegroundColor = ConsoleColor.Cyan;
189     Console.WriteLine("West to: ");
190     // Call the flood fill method to the west
191     board = FloodFill(board, row, col - 1);
192
193     // Print a message indicating the next flood fill direction
194     Console.ForegroundColor = ConsoleColor.Yellow;
195     Console.WriteLine("East to: ");
196     // Call the flood fill method to the east
197     board = FloodFill(board, row, col + 1);
198
199     // Print a message indicating the next flood fill direction
200     Console.ForegroundColor = ConsoleColor.Magenta;
201     Console.WriteLine("South to: ");
202     // Call the flood fill method to the south
203     board = FloodFill(board, row + 1, col);
204
205     // Print a message indicating the next flood fill direction
206     Console.ForegroundColor = ConsoleColor.Blue;
207     Console.WriteLine("North to: ");
208     // Call the flood fill method to the north
209     board = FloodFill(board, row - 1, col);
210
211     // Print a message indicating the next flood fill direction
212     Console.ForegroundColor = ConsoleColor.Cyan;
213     Console.WriteLine("NorthWest to: ");
214     // Call the flood fill method to the west
215     board = FloodFill(board, row - 1, col - 1);
216
217     // Print a message indicating the next flood fill direction
218     Console.ForegroundColor = ConsoleColor.Yellow;
219     Console.WriteLine("NorthEast to: ");
220     // Call the flood fill method to the east
221     board = FloodFill(board, row - 1, col + 1);
222
223     // Print a message indicating the next flood fill direction
224     Console.ForegroundColor = ConsoleColor.Magenta;
225     Console.WriteLine("SouthWest to: ");
226     // Call the flood fill method to the south
227     board = FloodFill(board, row + 1, col - 1);
228
229     // Print a message indicating the next flood fill direction
230     Console.ForegroundColor = ConsoleColor.Blue;
231     Console.WriteLine("SouthEast to: ");
232     // Call the flood fill method to the north
233     board = FloodFill(board, row + 1, col + 1);
234
235     // Return the board
236     return board;
237 } // End of FloodFill method

```

Figure 4-21: Code changes to flood fill for challenge one and three in the calling different directions section.

```

43
44     /// <summary>
45     /// Create shape to place on the board
46     /// </summary>
47     /// </summary>
48     public void PlaceShapes()
49     {
50         // Declare and initialize
51         // Random object to generate numbers
52         Random random = new Random();
53         int shapeSize = Size / 3, row = 0, col = 0;
54
55         // Create three shapes
56         for (int shapes = 0; shapes < NumShapes; shapes++)
57         {
58             // Generate the row and col for the
59             // top left corner of the square
60             row = random.Next(0, Size - shapeSize + 1);
61             col = random.Next(0, Size - shapeSize + 1);
62             for (int offset = 0; offset < shapeSize; offset++)
63             {
64                 // Top Wall
65                 Grid[row + offset, col + offset].Contents = "W";
66                 // Left Wall
67                 Grid[row + offset, col].Contents = "W";
68                 // Bottom Wall
69                 try
70                 {
71                     Grid[row + shapeSize - 1 + offset, col + offset].Contents = "W";
72                 }
73                 catch (IndexOutOfRangeException e)
74                 {
75                     // catch out of bounds and continue to next iteration
76                     continue;
77                 }
78                 // Right Wall
79                 Grid[row + offset + shapeSize - 1, col + shapeSize - 1].Contents = "W";
80             }
81         } // End of PlaceShapes method
82
83     }

```

Figure 4-22: Place shapes modifications for making a different shape type

Activity 4 Prep. Flow Chart

/*
* Chad Galloway
* CST - 250 Programming in C# II
* 11/16/2025
* Pizza Order System
* Activity 3 week 4 prep
* References:
*/

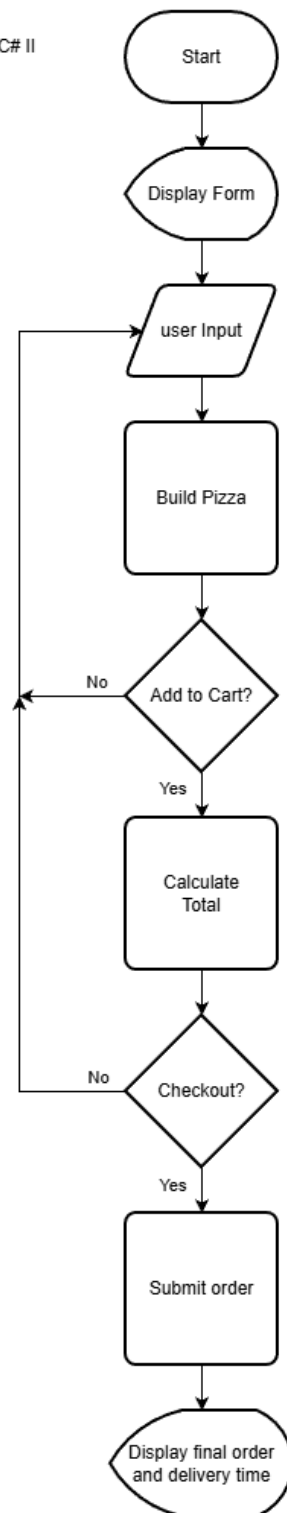


Figure 5-1: Flow chart of Activity 4.

```
/*
 * Chad Galloway
 * CST - 250 Programming in C# II
 * 11/16/2025
 * Pizza Order System
 * Activity 3 week 4 prep
 * References:
 */
```

